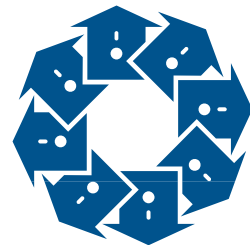




# JDBC Driver Guide

For use with c-treeSQL Server

This manual gives an overview of the JDBC (Java Database Connectivity) interface and the c-treeSQL JDBC Driver. It describes how to set up and use the driver and details the driver's support for the JDBC interface. The JDBC Driver provides access to c-treeSQL environments from applications that support JDBC.



**FairCom**<sup>®</sup>

Copyright © 1992-2004 FairCom Corporation All rights reserved.

Portions © 1987-2004 Dharma Systems, Inc. All rights reserved.

Eleventh Edition, First printing: September 2003

Information in this document is subject to change without notice.

No part of this publication may be stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise without the prior written permission of FairCom Corporation.

Printed in the United States of America.

FairCom welcomes your comments on this document and the software it describes. Send comments to:

Documentation Comments

FairCom Corporation

2100 Forum Blvd., Suite C

Columbia, MO 65203

Phone: 573-445-6833

Fax: 573-445-9698

Electronic Mail: [support@faircom.com](mailto:support@faircom.com)

Web Page: <http://www.faircom.com>

c-tree, c-tree Plus, r-tree, the circular disk logo, and FairCom are registered trademarks of the FairCom Corporation.

c-treeSQL, c-treeSQL ODBC, c-treeSQL ODBC SDK, c-treeVCL/CLX, c-tree ODBC Driver, c-tree Crystal Reports Driver, and c-treeDBX are trademarks of FairCom Corporation.

The following are third-party trademarks:

DBstore is a trademark of Dharma Systems, Inc.

Microsoft, MS-DOS, Windows, Windows NT, and Windows XP are registered trademarks of Microsoft Corporation.

Java, Java Development Kit, Solaris, SPARC, SunOS, and SunSoft are trademarks of Sun Microsystems, Inc.

Macintosh and MacOS are trademarks licensed to Apple Computer Co.

IBM and AIX are registered trademarks of International Business Machines Corp.

HP-UX is a registered trademark of Hewlett-Packard Company.

All other trademarks, trade names, company names, product names, and registered trademarks are the property of their respective holders

---

# Table of Contents

## Documentation Overview

|                                      |    |
|--------------------------------------|----|
| Purpose of This Manual . . . . .     | v  |
| Audience . . . . .                   | v  |
| Structure . . . . .                  | v  |
| Syntax Diagram Conventions . . . . . | vi |
| Related Documentation . . . . .      | vi |

## 1 Introduction

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| 1.1 Overview . . . . .                                                   | 1-1 |
| 1.2 JDBC Architecture . . . . .                                          | 1-1 |
| 1.3 Types of JDBC Drivers . . . . .                                      | 1-2 |
| 1.3.1 JDBC-ODBC Bridge Drivers . . . . .                                 | 1-2 |
| 1.3.2 Native-Method Drivers . . . . .                                    | 1-3 |
| 1.3.3 Network-Protocol All-Java Drivers . . . . .                        | 1-3 |
| 1.3.4 Native-Protocol All-Java Drivers (c-treeSQL JDBC Driver) . . . . . | 1-3 |
| 1.4 JDBC Compared to ODBC . . . . .                                      | 1-3 |

## 2 Quick Tour

|                                                                             |      |
|-----------------------------------------------------------------------------|------|
| 2.1 Introductory tutorial . . . . .                                         | 2-1  |
| 2.1.1 Init. . . . .                                                         | 2-2  |
| 2.1.2 Define . . . . .                                                      | 2-2  |
| 2.1.3 Manage . . . . .                                                      | 2-3  |
| 2.1.4 Done . . . . .                                                        | 2-4  |
| 2.1.5 Complete Introductory Tutorial Source Code . . . . .                  | 2-5  |
| 2.2 Relational Model and Indexing Tutorial . . . . .                        | 2-6  |
| 2.2.1 Init. . . . .                                                         | 2-6  |
| 2.2.2 Define . . . . .                                                      | 2-7  |
| 2.2.3 Manage . . . . .                                                      | 2-9  |
| 2.2.4 Done . . . . .                                                        | 2-10 |
| 2.2.5 Complete Relational Model and Indexing Tutorial Source Code . . . . . | 2-10 |
| 2.3 Locking Tutorial . . . . .                                              | 2-11 |
| 2.3.1 Init. . . . .                                                         | 2-11 |
| 2.3.2 Define . . . . .                                                      | 2-12 |
| 2.3.3 Manage . . . . .                                                      | 2-13 |
| 2.3.4 Done . . . . .                                                        | 2-15 |
| 2.3.5 Complete Locking Tutorial Source Code . . . . .                       | 2-16 |
| 2.4 Transaction Processing Tutorial . . . . .                               | 2-17 |
| 2.4.1 Init. . . . .                                                         | 2-17 |
| 2.4.2 Define . . . . .                                                      | 2-18 |
| 2.4.3 Manage . . . . .                                                      | 2-19 |
| 2.4.4 Done . . . . .                                                        | 2-21 |
| 2.4.5 Complete Transaction Processing Tutorial Source Code . . . . .        | 2-21 |

### 3 Basic JDBC Driver Operations

|                                                                                    |     |
|------------------------------------------------------------------------------------|-----|
| 3.1 Introduction . . . . .                                                         | 3-1 |
| 3.2 Required Java Environment . . . . .                                            | 3-1 |
| 3.3 Setting Up the JDBC Driver: Web Server . . . . .                               | 3-1 |
| 3.3.1 Copying JDBC Driver and Applet Class Files . . . . .                         | 3-2 |
| 3.3.2 Creating a Web Page That Invokes the Applet. . . . .                         | 3-2 |
| 3.4 Setting Up the JDBC Driver: Application Server . . . . .                       | 3-3 |
| 3.4.1 Copying JDBC Driver Files to the Application Server (If Necessary) . . . . . | 3-3 |
| 3.4.2 Setting Environment Variables. . . . .                                       | 3-3 |
| 3.4.3 Running the Sample Application . . . . .                                     | 3-4 |
| 3.5 Connecting to a Database . . . . .                                             | 3-4 |
| 3.5.1 Load the JDBC Driver Using Class.forName. . . . .                            | 3-4 |
| 3.5.2 Connect to the JDBC Driver Using DriverManager.getConnection . . . . .       | 3-4 |
| 3.5.2.1 Java URL Connection String . . . . .                                       | 3-5 |
| 3.5.2.2 User Authentication Detail . . . . .                                       | 3-5 |
| 3.5.3 An Example Connection. . . . .                                               | 3-5 |
| 3.5.4 Connection Pooling Support in JDBC . . . . .                                 | 3-6 |
| 3.5.4.1 javax.sql Package . . . . .                                                | 3-6 |
| 3.5.4.2 Using A DataSource Object To Make A Connection . . . . .                   | 3-6 |
| 3.5.4.3 Connection Pooling . . . . .                                               | 3-7 |
| 3.5.4.4 Implementation in Dharma JDBC Driver. . . . .                              | 3-7 |
| 3.6 Managing Transactions Explicitly to Improve Performance . . . . .              | 3-8 |

### 4 JDBC Conformance Notes

|                                                          |      |
|----------------------------------------------------------|------|
| 4.1 Supported Data Types. . . . .                        | 4-1  |
| 4.2 Return Values for DatabaseMetaData Methods . . . . . | 4-2  |
| 4.3 Error Messages . . . . .                             | 4-17 |

### A Glossary

|                     |     |
|---------------------|-----|
| A.1 Terms . . . . . | A-1 |
|---------------------|-----|

### B Tutorial Source Code

|                                                      |      |
|------------------------------------------------------|------|
| B.1 Introductory Tutorial . . . . .                  | B-1  |
| B.2 Relational Model and Indexing Tutorial . . . . . | B-4  |
| B.3 Locking Tutorial. . . . .                        | B-11 |
| B.4 Transaction Processing Tutorial. . . . .         | B-15 |

|                 |         |
|-----------------|---------|
| Index . . . . . | Index-i |
|-----------------|---------|

### List of Figures

|                                         |     |
|-----------------------------------------|-----|
| Figure 1-1: JDBC Architecture . . . . . | 1-2 |
|-----------------------------------------|-----|

### List of Tables

|                                                                |     |
|----------------------------------------------------------------|-----|
| Table 4-1: Mapping Between Java and JDBC Data Types . . . . .  | 4-1 |
| Table 4-2: Mapping Between JDBC and Java Data Types . . . . .  | 4-1 |
| Table 4-3: Return Values for DatabaseMetaData Methods. . . . . | 4-3 |

### List of Examples

---

Example 3-1: Loading the JDBC Driver and Connecting to a Database..... 3-5

Example 4-1: Getting Driver Information Through DatabaseMetadata Methods ..... 4-2



---

# Documentation Overview

## PURPOSE OF THIS MANUAL

This manual gives an overview of the JDBC (Java Database Connectivity) interface and the c-treeSQL JDBC Driver. It describes how to set up and use the driver and details the driver's support for the JDBC interface. The JDBC Driver provides access to the c-treeSQL environments from applications that support JDBC.

## AUDIENCE

This manual is directed towards application programmers writing database applications using c-treeSQL. It assumes knowledge of the Java™ programming language.

## STRUCTURE

This manual contains the following chapters:

|                   |                                                                                                                                                    |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Chapter 1</b>  | Introduces the c-treeSQL JDBC driver and describes how it works.                                                                                   |
| <b>Chapter 2</b>  | Includes tutorials for using the c-treeSQL JDBC driver.                                                                                            |
| <b>Chapter 3</b>  | Describes basic JDBC Driver operations: required software, setup, using the sample program, connecting to databases, and managing transactions.    |
| <b>Chapter 4</b>  | Details the information returned by the JDBC driver to the DatabaseMetadata methods and lists the supported SQL and corresponding Java data types. |
| <b>Appendix A</b> | Contains a glossary of terms you should know.                                                                                                      |
| <b>Appendix B</b> | Includes full source code for the JDBC tutorials.                                                                                                  |

## SYNTAX DIAGRAM CONVENTIONS

|           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UPPERCASE | Uppercase type denotes reserved words. You must include reserved words in statements, but they can be upper or lower case.                                                                                                                                                                                                                                                                                                                                                                     |
| lowercase | Lowercase type denotes either user-supplied elements or names of other syntax diagrams. User-supplied elements include names of tables, host-language variables, expressions, and literals. Syntax diagrams can refer to each other by name. If a diagram is named, the name appears in lowercase type above and to the left of the diagram, followed by a double-colon (for example, <code>privilege ::</code> ). The name of that diagram appears in lowercase in diagrams that refer to it. |
| { }       | Braces denote a choice among mandatory elements. They enclose a set of options, separated by vertical bars (   ). You must choose at least one of the options.                                                                                                                                                                                                                                                                                                                                 |
| [ ]       | Brackets denote an optional element or a choice among optional elements.                                                                                                                                                                                                                                                                                                                                                                                                                       |
|           | Vertical bars separate a set of options.                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| ...       | A horizontal ellipsis denotes that the preceding element can optionally be repeated any number of times.                                                                                                                                                                                                                                                                                                                                                                                       |
| ( ) , ;   | Parentheses and other punctuation marks are required elements. Enter them as shown in syntax diagrams.                                                                                                                                                                                                                                                                                                                                                                                         |

## RELATED DOCUMENTATION

Refer to the following manuals for more information:

|                                                 |                                                                                                                        |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <i>c-treeSQL Reference Manual</i>               | Describes the syntax and semantics of statements and language elements for the c-treeSQL interface.                    |
| <i>c-treeSQL ISQL and Tools Reference Guide</i> | Describes the c-treeSQL interactive SQL utility, ISQL, and other database administration utilities.                    |
| <i>c-treeSQL Embedded SQL User's Guide</i>      | Describes how to develop host language programs containing embedded SQL statements that access c-treeSQL environments. |
| <i>c-treeSQL ODBC Driver Guide</i>              | Describes c-treeSQL support for the ODBC interface and how to configure the c-treeSQL ODBC Driver.                     |



|                                                               |                                                                                                                                                                                                                                                                    |
|---------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>c-treeSQL Guide to Java Stored Procedures and Triggers</i> | Describes how to write and use Java stored procedures and triggers—Java routines which contain SQL statements and are stored in a database. ODBC, JDBC, and SQL applications call stored procedures, while triggers are invoked automatically by database updates. |
| <i>c-tree Plus Quick Start and Product Overview Guide</i>     | Describes the installation process, how to get started, and recommendations for c-tree Plus ISAM/Low-Level, c-treeDB, c-treeVCL/CLX, c-tree Server, and c-treeSQL Server.                                                                                          |

## Java Information

James Gosling & Henry McGilton

*"The Java™ Language Environment: A White Paper"*

<http://java.sun.com/docs/white/langenv/>

Mary Campione and Kathy Walrath

*The Java™ Tutorial*

<http://java.sun.com/docs/books/tutorial/index.html>

*Java™ Platform 1.1.3 Core API*

<http://www.javasoft.com:80/products/jdk/1.1/docs/api/packages.html>

Gary Cornell and Cay Horstmann

*Core Java 1.1 Volume 1: The Fundamentals*

Prentice Hall, Upper Saddle River, NJ, 1997

## JDBC Information

JavaSoft's JDBC home page

<http://java.sun.com/products/jdbc/>

George Reese

*Database Programming with JDBC and Java*

O'Reilly and Associates, Sebastopol, CA, 1997

### **ODBC Information**

*Microsoft ODBC Programmer's Reference, Version 3.0* - Describes the ODBC interface, its features, and how applications use it.

## Introduction

### 1.1 OVERVIEW

The c-treeSQL JDBC Driver provides access to c-treeSQL environments from applications that support the JDBC 2.0 API.

JDBC allows applications to connect to any database using the same set of Java interfaces. Those interfaces allow programs to embed standard Structured Query Language (SQL) statements that update and retrieve data in the database.

Because the Java interfaces and SQL syntax are independent of any particular database implementation, JDBC makes it feasible for applications to connect to different database environments without any modification.

### 1.2 JDBC ARCHITECTURE

JDBC insulates Java applications from variations in database access implementations through the JDBC API, a set of class libraries distributed as a standard part of core Java. Instead of using calls to vendor-specific interfaces, JDBC applications use the JDBC API.

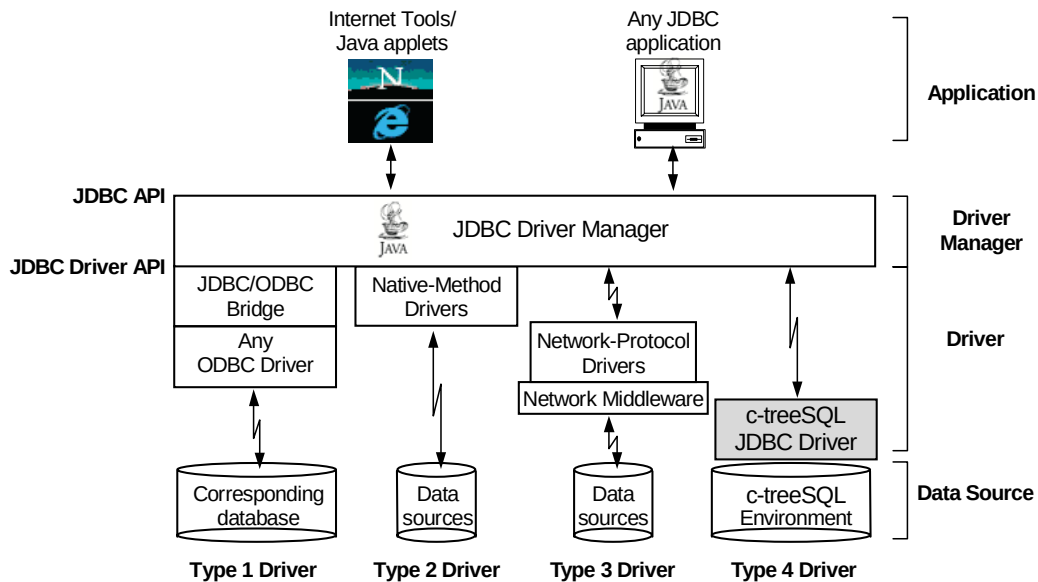
The JDBC API is distributed as the package *java.sql* and is included with the JavaSoft JDK (Version 1.4 or later), so any environment that supports a recent Java compiler can be used to develop JDBC applications.

Calls to the JDBC API are managed by the JDBC driver manager. The JDBC driver manager can support multiple drivers connecting to different databases. When an application tries to connect to a particular database, the driver manager loads the appropriate JDBC driver and routes subsequent calls through the driver.

A JDBC driver is a database-specific software that receives calls from the JDBC driver manager, translates them into a form that the database can process, and returns data to the application.

The following figure shows the different components of the JDBC architecture.

Figure 1-1: JDBC Architecture



## 1.3 TYPES OF JDBC DRIVERS

JDBC drivers can either be entirely written in Java so that they can be downloaded as part of an applet, or they can be implemented using native methods to bridge to existing database access libraries.

JavaSoft defines four different types of JDBC drivers, as noted in the previous figure and outlined in the following sections.

### 1.3.1 JDBC-ODBC Bridge Drivers

Type 1 drivers translate calls to JDBC methods into calls to Microsoft Open Database Connectivity (ODBC) functions. Bridge drivers allow JDBC applications immediate access to database connectivity provided by the existing array of ODBC drivers.

Both the JavaSoft JDK and Microsoft Java SDK include JDBC-ODBC bridge drivers.

ODBC architecture requires that the ODBC driver manager and (typically) the ODBC drivers themselves be loaded on each client system. The requirement for software resident on client systems means that JDBC-ODBC bridge drivers will not work with Java applets run from an Internet browser. Browsers do not allow applets to run another program on the client to which they are downloaded. (In general, JDBC-ODBC bridge drivers will not work in environments that restrict Java applications from reading and writing files or running other programs.)

JDBC-ODBC bridge drivers are still useful in corporate networks, or for use by application server code written in Java in a 3-tier architecture. In such an environment, the application server has intermediary software, such as Blue Lobster's Aptivity, that receives requests from browsers and other Internet applications. The intermediary software in turn calls the JDBC driver manager when it receives a database request.

### **1.3.2 Native-Method Drivers**

Type 2 drivers contain Java code that calls "native" C or C++ methods already implemented by database vendors.

Like an ODBC driver, a native-method driver must be installed on each client or server that uses it, and thus has the same limitations as the JDBC-ODBC bridge drivers. A typical use of native-method drivers is on application servers.

### **1.3.3 Network-Protocol All-Java Drivers**

Type 3 drivers are completely written in Java. They translate JDBC calls into a database-independent network protocol which is in turn translated to a DBMS protocol by middleware on a network server.

This type of driver can thus connect many Java clients to many different databases. The specific protocol used depends on the vendor.

Type 3 drivers are the most flexible since they do not require any driver software resident on client systems and can allow a single driver to provide access to multiple databases.

### **1.3.4 Native-Protocol All-Java Drivers (c-treeSQL JDBC Driver)**

Type 4 drivers are also written completely in Java, but do not rely on middleware. They convert JDBC calls directly into the network protocol used by a particular database. This approach allows a direct call from the client system to the database server. Also, since there is no client-resident software, it also is practical for Internet applications.

Type 4 drivers provide the best performance.

The c-treeSQL JDBC Driver is a Type 4 driver.

## **1.4 JDBC COMPARED TO ODBC**

Generally speaking, JDBC is to Java what Microsoft's Open Database Connectivity (ODBC) interface is to the C language. Both JDBC and ODBC:

- Provide a vendor-independent API that allows the same application to connect to different vendors' databases and retrieve and update data using standard SQL statements.
- Adopt the architecture of imposing a driver manager between applications and vendor-supplied drivers that translate between the standard API and a vendor's proprietary implementation.

- Are based on the X/Open SQL call-level interface specification.

JDBC proponents cite these advantages of JDBC over ODBC:

- JDBC applications enjoy the platform-independence of Java, which lends itself to Internet applications. ODBC applications must, at a minimum, be recompiled to run on a different operating-system/hardware combination.
- JDBC does not require software on each client system, which also recommends it for Internet applications.
- JDBC is much simpler and easier to learn than ODBC.
- JDBC is not primarily targeted for PC application development, which makes for faster implementation outside the Windows environment.

---

# Chapter 2

## Quick Tour

### 2.1 INTRODUCTORY TUTORIAL

#### JDBC\_Tutorial1.java

This introductory tutorial will rapidly take you through the basic use of the powerful FairCom JDBC database interface. The FairCom JDBC Driver is a native type 4 driver written completely in Java. JDBC calls are converted directly into the network protocol allowing a direct call from a client system to the database server.

By no means does this introduction cover the full scope, detail, or flexibility the FairCom JDBC driver offers. It does however provide a quick glimpse into the advantages of implementing a database solution using Java and JDBC.

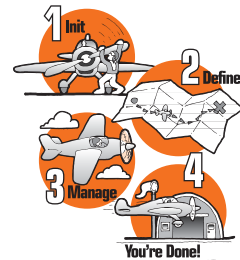
This example, like all others in this set of documentation, will take the creation and use of a database and fit it into a simple four step flow of initialization, definition, management, and completion. (Init, define, manage, and your're done!)

To simplify, let's create a simple mainline below:

```
/* global variables */
static Connection conn;
static Statement stmt;
public static void main(String[] args) {

    init ();
    define ();
    manage ();
    done ();
}
```

Now let's break into the four areas.



### 2.1.1 Init

In the client/server database model the client is required to open a connection to a Database by providing the Server with a user name, password and the database name. The initialize() function invokes a series of JDBC methods required to select a driver and create a connection. The following JDBC calls provide the functionality to "Initialize".



- `Class.forName` — Specifies a driver to be loaded
- `DriverManager.getConnection` — Makes a database connection through the driver manager
- `Connection.createStatement` — Creates a statement object for SQL submits

Below is the code for INIT:

```
private static void initialize ()
{
    System.out.println("INIT");
    try {
        // Load the driver class
        Class.forName ("ctree.jdbc.ctreeDriver");

        conn = DriverManager.getConnection
            ("jdbc:ctree:6597@localhost:myDatabase", "ADMIN", "ADMIN");
        // Create a SQL statement
        stmt = conn.createStatement();
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}
```

### 2.1.2 Define

In this case define() consists of the CREATE TABLE statement. This is done in a single JDBC call in which specific fields are defined. The following SQL syntax provides the functionality for the define phase:



- `Statement.execute` — Submits SQL statement for direct execution

Below is the code for DEFINE:

```
private static void define ()
{
    String tblCreate = "CREATE TABLE CUSTMAST (" +
        "cm_custnum    VARCHAR(5), " +
        "cm_custzip    VARCHAR(10), " +
        "cm_custstate  VARCHAR(3), " +
        "cm_custrating VARCHAR(2), " +
        "cm_custname   VARCHAR(48), " +
```



```

        "cm_custaddress VARCHAR(48), " +
        "cm_custcity   VARCHAR(48))";

    try {
        stmt.executeUpdate(tblCreate);
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

```

## 2.1.3 Manage

This step provides data management functionality for the application. We will simply add records to a table and then get and display those records. Then a simple record deletion is performed. The following JDBC calls provide the functionality to manipulate the records in our table.



- `Statement.executeUpdate` — Submits an SQL statement for direct execution
- `Statement.executeQuery` — Submits an SQL statement expecting a result set

Below is the code for MANAGE:

```

private static void manage ()
{
    System.out.println("MANAGE");
    Delete_Records ();    // start fresh
    Add_Records ();
    Display_Records ();
    Delete_Records ();
}

private static void Add_Records ()
{
    System.out.println("\tAdd Records");
    String sqlInsert = "INSERT INTO custmast VALUES ";
    String RecordData[] = {
        "('1000','92867','CA','1','Bryan Williams','2999 Regency','Orange')",
        "('1001','61434','CT','1','Michael Jordan','13 Main','Harford')",
        "('1002','73677','GA','1','Joshua Brown','4356 Cambridge','Atlanta')",
        "('1003','10034','MO','1','Keyon Dooling','19771 Park Avenue',
        'Columbia')" };

    try {
        /* Loop inserting 1 record at a time */
        for (int count = 0; count < 4; count++)
        {
            stmt.executeUpdate(sqlInsert + RecordData[count]);
        }
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

```

```
    }  
}  
  
private static void Display_Records ()  
{  
    System.out.println("\tDisplay Records");  
  
    try {  
        /* Execute a query */  
        ResultSet rs = stmt.executeQuery ("select * from custmast");  
  
        /* Fetch the data from the query result set */  
        while (rs.next()) {  
            System.out.println(rs.getString(1) + "    " + rs.getString(5));  
        }  
    } catch (Exception e) {  
        System.out.println("Exception: " + e.getMessage());  
    }  
}  
  
private static void Delete_Records ()  
{  
    System.out.println("\tDelete Records");  
  
    try {  
        stmt.executeUpdate("DELETE FROM custmast");  
    } catch (Exception e) {  
        System.out.println("Exception: " + e.getMessage());  
    }  
}
```

### 2.1.4 Done

When a client application has completed operations with the server, it must release resources by disconnecting from the database. The Connection and Statement classes provide a close() member method to handle deallocation of resources.

- Connection.close
- Statement.close

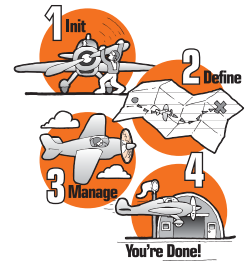
Below is the code for DONE:

```
private static void done ()  
{  
    System.out.println("DONE");  
    try {  
        stmt.close();  
        conn.close();  
    } catch (Exception e) {  
        System.out.println("Exception: " + e.getMessage());  
    }  
}
```



### 2.1.5 Complete Introductory Tutorial Source Code

Complete source code for the introductory tutorial can be found in [Appendix B "Tutorial Source Code" on page B-1](#).



## 2.2 RELATIONAL MODEL AND INDEXING TUTORIAL

### JDBC\_Tutorial2.java

This intermediate tutorial will advance the concepts introduced in the first tutorial by expanding the number of tables and building a relational model. This tutorial will walk you through defining an index for each table, demonstrating the power of indexes in a relational model using a few simple API calls.

This example, like all others in this set of documentation, will take the creation and use of a database and fit it into a simple four step flow of initialization, definition, management, and completion. (Init, define, manage, and your're done!)

To simplify, let's create a simple mainline below:

```
/* global variables */
static Connection conn;
static Statement stmt;
static Statement stmt1;
static Statement stmt2;
static Statement stmt3;

public static void main(String[] args) {
    initialize ();
    define ();
    manage ();
    done ();
}
```

Now let's break into the four areas:

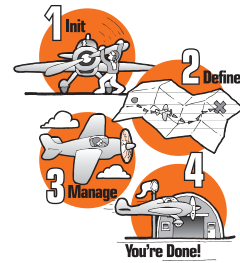
### 2.2.1 Init

In the client/server database model the client is required to open a connection to a Database by providing the Server with a user name, password and the database name. The initialize() method invokes a series of JDBC methods required to select a driver and create a connection. The following JDBC calls provide the functionality to "Initialize".

- `Class.forName` — Specifies a driver to be loaded
- `DriverManager.getConnection` — Makes a database connection through the driver manager
- `Connection.createStatement` — Creates statement object for SQL submits
- `Connection.setAutoCommit` — Controls the operation of transaction processing

Below is the code for INIT:

```
private static void initialize ()
```



```

{
    System.out.println("INIT");
    try {
        // Load the driver class
        Class.forName ("ctree.jdbc.ctreeDriver");

        conn = DriverManager.getConnection
            ("jdbc:ctree:6597@localhost:myDatabase",
             "ADMIN", "ADMIN");

        // Create a SQL statement
        stmt = conn.createStatement();
        stmt1 = conn.createStatement();
        stmt2 = conn.createStatement();
        stmt3 = conn.createStatement();

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

```

### 2.2.2 Define

In this case define() consists of defining a record made up of fields and an associated index. An index is created by defining 1 or more segments of the record. Typically a segment is defined as a field in the record as in this example.



#### ***Relational Database***

This process of defining tables and indices is creating a relational database. In this example there are 4 tables being defined as depicted in the drawing below. The index segments are shown in bold italics.

- **OrderList** — A table of records consisting of a list of orders
- **OrderItem** — A table of records consisting of specific items associated with an order
- **ItemMaster** — A table of records consisting of information about items
- **CustomerMaster** — A table of records consisting of specific info related to each customer

Each order (ordernum) in the ***orderlist*** table will contain 1 or more items (itemnum) in the ***orderitem*** table. Each item will have a corresponding definition (weight, price, description) in

the *itemmast* table. An order is related to a specific customer (custnum) in the *custmast* table which contains information about each customer.

| OrderList                                                | OrderItem                                                | ItemMaster                                | CustMast                                                            |
|----------------------------------------------------------|----------------------------------------------------------|-------------------------------------------|---------------------------------------------------------------------|
| orderdate<br>promdate<br><b>ordnum</b><br><b>custnum</b> | <b>ordnum</b><br><b>seqnumber</b><br>quantity<br>itemnum | weight<br>price<br><b>itemnum</b><br>desc | <b>custnum</b><br>zip<br>state<br>rating<br>name<br>address<br>city |

- Statement.executeUpdate — Submits SQL statement for direct execution

Below is the code for DEFINE:

```
private static void define ()
{
    System.out.println("DEFINE");

    System.out.println("\tCreate Tables");

    /* a list of orders by customer */
    Create_OrderList_Table();

    /* specific items regarding orders */
    Create_OrderItem_Table();

    /* information about product items */
    Create_ItemMaster_Table();

    /* information about customers */
    Create_CustomerMaster_Table();
}
```

The following method shows an example of the simple steps involved in the creation of a table.

```
private static void Create_OrderList_Table ()
{
    String tblCreate = "CREATE TABLE orderlist (" +
        "ol_orderdate DATE, " +
        "ol_promdate DATE, " +
        "ol_ordernum VARCHAR(7), " +
        "ol_custnum VARCHAR(4))";
    try {
        stmt.executeUpdate(tblCreate);
        stmt.executeUpdate("CREATE INDEX custorder ON
            orderlist (ol_ordernum, ol_custnum)");
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}
```

```
}
```

### 2.2.3 Manage

This step provides data management functionality for the application. In this example we will simply add records to the tables, process records in the form of a query, and finally display the results of the query.

This process involves fetching from the *orderlist* table and, for each record, fetching related records from the *orderitems* table based on the index *ordernum*. The result is a recordset that amounts to a list of items associated with an order. The output of the manage step is the total price of all items for each order and the customer related to the order.



The following SQL syntax provides the functionality for the manage phase:

- `Statement.executeUpdate` — Submits an SQL statement for direct execution
- `Statement.executeQuery` — Submits an SQL statement expecting a result set
- `Connection.commit` — Makes changes to the database permanent
- `Connection.rollback` — Cancels all changes to the database since the last commit

The following snippet of code implements standard SQL using the JDBC API to find a specific record. In this case the search argument is *custnum*. The desired result is the record associated with *custnum* from the Customer Master table. The Java `ResultSet` object `rsCustMast`, maintains a cursor to the the unique record which allows accesss to individual fields. This is how the name related to the order is retrieved.

```
ResultSet rsCustMast = stmt1.executeQuery ("SELECT * " +
    "FROM custmast " +
    "WHERE cm_custnum = '" +
    rsOrderList.getString(4) + "'");
```

The following snippet of code demonstrates the use of a search argument on an index in a select statement. This select statement will narrow a record set down to just the records matching a specific order number in the *orderitems* table. This allows processing to loop on each record related to a specific order. Each unique record specifies an *itemnum* associated with this order. *itemnum* is then used as an argument to `SELECT` the price and quantity of each item from the *itemmast* table. Member methods of the `ResultSet` Object, `getShort()` and `getFloat()` access the fields price and quantity from the record to arrive at a cumulative price of the order.

```
ResultSet rsOrderItem = stmt2.executeQuery ("SELECT * " +
    "FROM orderitems " +
    "WHERE oi_ordernum = '" +
    rsOrderList.getString(3) + "'");
totalPrice = 0;
while (rsOrderItem.next()) {
```

```
ResultSet rsItemMast = stmt3.executeQuery ("SELECT * " +
    FROM itemmast " +
    "WHERE im_itemnum = '" +
    rsOrderItem.getString(4) + "'");
rsItemMast.next();
totalPrice += (rsItemMast.getFloat(2) *
    rsOrderItem.getShort(3));
rsItemMast.close();
}
System.out.println("\t " + rsCustMast.getString(5) + " " +
    totalPrice);
rsOrderItem.close();
rsCustMast.close();
```

## 2.2.4 Done

When a client application has completed operations with the server, it must release resources by disconnecting from the database. The Connection and Statement classes provide a close() member method to handle deallocation of resources.

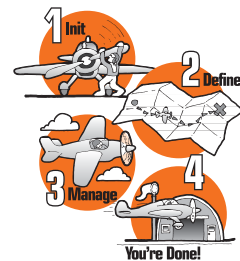
- Connection.close
- Statement.close

Below is the code for DONE:

```
private static void done ()
{
    System.out.println("DONE");
    try {
        stmt.close();
        stmt1.close();
        stmt2.close();
        stmt3.close();
        conn.close();
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}
```

## 2.2.5 Complete Relational Model and Indexing Tutorial Source Code

Complete source code for the relational model and indexing tutorial can be found in [Appendix B "Tutorial Source Code" on page B-1](#).





## 2.3 LOCKING TUTORIAL

### JDBC\_Tutorial3.java

This tutorial will introduce the concept of locking. The functionality for this tutorial focuses on adding records, then updating a single record to the customer master table. The application will pause while a LOCK is placed on a record. Another instance of this application will be launched, which will block, waiting on the lock held by the first instance. Pressing the <Enter> key will enable the first instance to proceed. This will result in removing the lock thereby allowing the second instance to continue execution. Launching two processes provides a visual demonstration of the effects of locking and a basis for experimentation on your own.

This example, like all others in this set of documentation, will take the creation and use of a database and fit it into a simple four step flow of initialization, definition, management, and completion. (Init, define, manage, and you're done!)

To simplify, let's create a simple mainline below:

```
/* global variables */
static Connection conn;
static Statement stmt;
static Statement stmt1;
static Statement stmt2;

public static void main(String[] args) {

    initialize ();
    define ();
    manage ();
    done ();
}
```

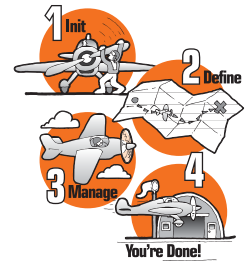
Now let's break into the four areas.

### 2.3.1 Init

In the client/server database model the client is required to open a connection to a Database by providing the Server with a user name, password and the database name. The initialize() method invokes a series of JDBC methods required to select a driver and create a connection.

The following JDBC calls provide the functionality to "Initialize".

- Class.forName — Specifies a driver to be loaded
- DriverManager.getConnection — Makes a database connection through the driver manager
- Connection.createStatement — Creates statement object for SQL submits



- `Connection.setAutoCommit` — Controls the operation of transaction processing

Below is the code for INIT:

```
private static void initialize ()
{
    System.out.println("INIT");
    try {
        // Load the driver class
        Class.forName ("ctree.jdbc.ctreeDriver");

        conn = DriverManager.getConnection
            ("jdbc:ctree:6597@localhost:myDatabase",
            "ADMIN", "ADMIN");

        // disable each SQL statement as a single transaction
        conn.setAutoCommit(false);

        // Create a SQL statement
        stmt = conn.createStatement();
        stmt1 = conn.createStatement();
        stmt2 = conn.createStatement();

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}
```

### 2.3.2 Define

In this case define() consists of the CREATE TABLE statement. This is done in a single JDBC call in which specific fields are defined. This tutorial operates solely on the customer master table in order to focus on locking.

The following JDBC calls provide the functionality for the define phase:

- `Statement.executeUpdate` — Submits an SQL statement for direct execution

Below is the code for DEFINE:

```
private static void define ()
{
    System.out.println("DEFINE");

    // Create customer master table
    String tblCreate = "CREATE TABLE custmast (" +
        "cm_custnum VARCHAR(5), " +
        "cm_zip VARCHAR(10), " +
        "cm_state VARCHAR(3), " +
        "cm_rating VARCHAR(2), " +
        "cm_name VARCHAR(48), " +
```



```

        "cm_address VARCHAR(48), " +
        "cm_city VARCHAR(48))";

    try {
        stmt.executeUpdate(tblCreate);
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

```

### 2.3.3 Manage

This step provides data management functionality for the application. In this example we will operate on the customer master table by first deleting all records, then adding, updating and finally displaying the contents of the table. Deleting all records will ensure a clean starting point. The adds will populate the table allowing record manipulation. An important thing to remember is that locking is inherent at the SQL level. This means that when one process has a record selected for update, it is locked even though there is no explicit SQL syntax.



The intention of this tutorial is to provide a visual demonstration of the effect of locking. This requires the user to launch two instances of JDBC\_Tutorial3.exe. The first process will execute to a point and then pause waiting for a keystroke. Launch the second process, it will encounter a lock on a record it is trying to delete and will block.

The first process has the record associated with customer number 1003 locked. Meanwhile the second process has attempted to delete the contents of the customer master table and has blocked trying to lock the table. At this point both processes are effectively blocked. The first process is holding the lock waiting on a keystroke and the second is waiting to grab the lock.

Pressing the <Enter> key in the first process will resume execution resulting in record 1003 being unlocked. If the first process is allowed to continue execution then it will display the updated (uppercased customer name) record associated with 1003. Otherwise if the first process is preempted there are many possibilities, including the updated record will be deleted by the time the contents of the table are displayed.

The following graphic depicts 2 consoles executing JDBC\_Tutorial3. Both are blocked as described above. The first process is waiting for a keystroke and the second process is waiting for the lock held by the first process.

```

C:\Faircom>java JDBC_Tutorial3
INIT
DEFINE
MANAGE

Delete Records
Add Records

Display CUSTMAST table
1000 Bryan Williams
1001 Michael Jordan
1002 Joshua Brown
1003 Keyon Dooling
Update
Press <Enter>

C:\Tutorials\JDBC>java JDBC_Tutorial3
INIT
DEFINE
Exception: Table/View/Synonym already exists
MANAGE

Delete Records
  
```

The following graphic depicts the effects of pressing the <Enter> key in the first process. The result is a flurry of activity. Visually it is difficult to discern the sequence of events. Based on the results, the first process unlocked allowing the second process to regain control. The second process deleted the updated record prior to the first process getting a chance to display the contents of the updated record.

```

C:\Faircom>java JDBC_Tutorial3
Delete Records
Add Records

Display CUSTMAST table
1000 Bryan Williams
1001 Michael Jordan
1002 Joshua Brown
1003 Keyon Dooling
Update
Press <Enter>

DONE

Display CUSTMAST table

C:\Tutorials\JDBC>java JDBC_Tutorial3
INIT
DEFINE
Exception: Table/View/Synonym already exists
MANAGE

Delete Records
Add Records

Display CUSTMAST table
1000 Bryan Williams
1001 Michael Jordan
1002 Joshua Brown
1003 Keyon Dooling
Update
Press <Enter>
  
```

The following SQL syntax provides the functionality for the manage phase:

- Statement.executeUpdate — Submits an SQL statement for direct execution
- Statement.executeQuery — Submits an SQL statement expecting a result set
- Connection.commit — Makes changes to the database permanent

The following method controls the manage functionality related to locking. The table is cleared of any existing records due to a previous run or from a prior tutorial. This allows for a clear starting point for demonstration purposes. The table is populated and displayed allowing for an update to a populated record. Then as discussed previously, multiple processes interact allowing for the effect of locking to be visualized.

```

private static void manage ()
{
    System.out.println("MANAGE");

    Delete_Records ();

    System.out.println("\tAdd Records");
    Add_CustomerMaster_Records();

    Display_Records();    /* show contents of table */

    Update_CustomerMaster_Record();
    Display_Records();    /* show contents of table */
}

```

This is the time to launch another instance of this application. It will execute into the Delete\_Records() method until it tries to access the locked record held by the first process. This will force the second process to yield until a keystroke is pressed in the first process.

```

private static void Update_CustomerMaster_Record()
{
    InputStreamReader isr = new InputStreamReader(System.in);
    BufferedReader stdin = new BufferedReader (isr);

    System.out.println("\tUpdate");

    try {
        stmt.executeUpdate("UPDATE custmast SET cm_name =
            'KEYON DOOLING' where cm_custnum = '1003'");
        System.out.println("\tPress <Enter>");
        String input = stdin.readLine();
        conn.commit();

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

```

### 2.3.4 Done

When a client application has completed operations with the server, it must release resources by disconnecting from the data-base. The Connection and Statement classes provide a close() member method to handle deallocation of resources.

- Connection.close
- Statement.close

Below is the code for DONE:

```

private static void done ()
{

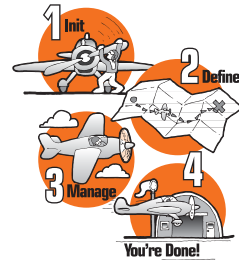
```



```
System.out.println("DONE");
try {
    stmt.close();
    stmt1.close();
    stmt2.close();
    conn.close();
} catch (Exception e) {
    System.out.println("Exception: " + e.getMessage());
}
}
```

### 2.3.5 Complete Locking Tutorial Source Code

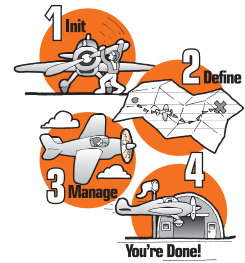
Complete source code for the locking tutorial can be found in [Appendix B "Tutorial Source Code" on page B-1](#).



## 2.4 TRANSACTION PROCESSING TUTORIAL

### JDBC\_Tutorial4.java

This tutorial will introduce the concept of transaction processing. It is based on the relational model of the previous tutorial. Records will be added to tables `orderlist` and `orderitems` as a single transaction. Therefore if any part of the transaction fails, the entire transaction will be abandoned. In this example a valid transaction requires valid item numbers that make up an order and a valid customer number associated with each order.



This example, like all others in this set of documentation, will take the creation and use of a database and fit it into a simple four step flow of initialization, definition, management, and completion. (Init, define, manage, and you're done!)

To simplify, let's create a simple mainline below:

```
/* global variables */
static Connection conn;
static Statement stmt;
static Statement stmt1;
static Statement stmt2;

public static void main(String[] args) {
    initialize ();
    define ();
    manage ();
    done ();
}
```

Now let's break into the four areas.

### 2.4.1 Init

In the client/server database model the client is required to open a connection to a Database by providing the Server with a user name, password and the database name. The `initialize()` method invokes a series of JDBC methods required to select a driver and create a connection. The following JDBC calls provide the functionality to "Initialize".



- `Class.forName` — Specifies a driver to be loaded
- `DriverManager.getConnection` — Makes a database connection through the driver manager
- `Connection.createStatement` — Creates statement object for SQL submits
- `Connection.setAutoCommit` — Controls the operation of transaction processing

Below is the code for INIT:

```
private static void initialize ()
```

```
{
    System.out.println("INIT");
    try {
        // Load the driver class
        Class.forName ("ctree.jdbc.ctreeDriver");

        conn = DriverManager.getConnection
            ("jdbc:ctree:6597@localhost:myDatabase",
            "ADMIN", "ADMIN");

        // disable each SQL statement as a single transaction
        conn.setAutoCommit(false);

        // Create a SQL statement
        stmt = conn.createStatement();
        stmt1 = conn.createStatement();
        stmt2 = conn.createStatement();

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}
```

### 2.4.2 Define

In this case define() consists of the CREATE TABLE statement. This is done in a single JDBC call in which specific fields are defined. The following SQL syntax provides the functionality for the define phase:

- `Statement.execute` — Submits SQL statement for direct execution

Below is the code for DEFINE:

```
private static void define ()
{
    System.out.println("DEFINE");

    /* a list of orders by customer */
    Create_OrderList_Table();

    /* specific items regarding orders */
    Create_OrderItem_Table();

    /* information about product items */
    Create_ItemMaster_Table();

    /* information about customers */
    Create_CustomerMaster_Table();
}
```

The following method shows an example of the simple steps involved in the creation of a table.





```

private static void Create_OrderList_Table ()
{
    String tblCreate = "CREATE TABLE orderlist (" +
        "ol_orderdate DATE, " +
        "ol_promdate DATE, " +
        "ol_ordnum VARCHAR(7), " +
        "ol_custnum VARCHAR(4))";
    try {
        stmt.executeUpdate(tblCreate);
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

```

### 2.4.3 Manage

This step provides data management functionality for the application. In this example we will add records to the itemmast and custmast tables, intended as static data. Then, sales orders will be processed as a transaction and "committed" or "rolled back" depending on the validity of the data. The final step will be to display the result of the transaction processing by dumping the contents of the orderlist and orderitems tables.



The following JDBC syntax provides the functionality for the manage phase:

- `Statement.executeUpdate` — Submits an SQL statement for direct execution
- `Statement.executeQuery` — Submits an SQL statement expecting a result set
- `Connection.commit` — Makes changes to the database permanent
- `Connection.rollback` — Cancels all changes to the database since the last commit

```

private static void manage ()
{
    System.out.println("MANAGE");

    Delete_Records ();

    Add_ItemMaster_Records();
    Add_CustomerMaster_Records();

    Add_Transactions();

    Display_OrderList ();
    Display_OrderItems ();
}

```

The following snippet of code implements standard SQL to insert the item records as part of the transaction. Each item is inserted in the table and verified against the itemmaster table. If

the verification is unsuccessful, the transaction is rolled back and no further processing is performed on the transaction.

```
while (ItemData[iItems][0].compareTo(OrderData[iOrders][2]) == 0)
{
    /* INSERT items associated with an order */
    sql = "INSERT INTO orderitems VALUES ( " +
        "'" + ItemData[iItems][0] + "'," +
        ItemData[iItems][1] + "," +
        ItemData[iItems][2] + "," +
        ItemData[iItems][3] + "')";
    stmt.executeUpdate(sql);

    // Does this itemnum exist in the master
    sql = "SELECT im_itemnum FROM itemmast WHERE im_itemnum = '" +
        ItemData[iItems][3] + "'";

    ResultSet rsItemMast = stmt.executeQuery (sql);

    try {
        if (!(rsItemMast.next()))
        {
            conn.rollback();
            bTransFail = true;
            break;
        }
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
    iItems++;
}
```

The following snippet of code implements standard SQL to INSERT the order portion of the transaction. Subsequently the customer number is verified with the customer master table. If the specific customer number is found then the entire transaction is committed. Otherwise the transaction is rolled back thus "undoing" the item INSERTS associated with this order transaction.

```
sql = "INSERT INTO orderlist VALUES ( " +
    "'" + OrderData[iOrders][0] + "'," +
    OrderData[iOrders][1] + "," +
    OrderData[iOrders][2] + "," +
    OrderData[iOrders][3] + "')";
stmt.executeUpdate(sql);

sql = "SELECT cm_custnum FROM custmast WHERE cm_custnum = '" +
    OrderData[iOrders][3] + "'";
ResultSet rsCustMast = stmt1.executeQuery (sql);
if (rsCustMast.next())
    conn.commit();
else
    conn.rollback();
```

### 2.4.4 Done

When a client application has completed operations with the server, it must release resources by disconnecting from the database. The Connection and Statement classes provide a close() member method to handle deallocation of resources.

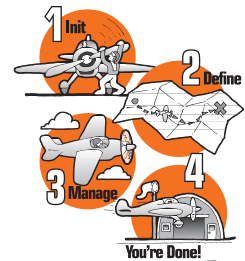
- Connection.close
- Statement.close

Below is the code for DONE:

```
private static void done ()
{
    System.out.println("DONE");
    try {
        stmt.close();
        stmt1.close();
        stmt2.close();
        conn.close();
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}
```

### 2.4.5 Complete Transaction Processing Tutorial Source Code

Complete source code for the transaction processing tutorial can be found in [Appendix B "Tutorial Source Code" on page B-1](#).





# Basic JDBC Driver Operations

### 3.1 INTRODUCTION

This chapter describes how to set up and get started using the c-treeSQL JDBC Driver.

**Note:** Descriptions in this section specify file and directory pathnames relative to the top-level c-treeSQL directory on the system where the libraries and executable were built. For instance, if the top-level c-treeSQL directory is *c:\ctree*, the relative pathname *classes* specifies the directory *c:\ctree\classes*.

### 3.2 REQUIRED JAVA ENVIRONMENT

You must have a supported Java development or runtime environment on each system that uses the JDBC Driver. The c-treeSQL JDBC Driver requires the following development (or compatible runtime) environments:

- On UNIX: JavaSoft JDK™ Version 1.4
- On Windows NT or Windows 2000: JavaSoft JDK™ Version 1.4 or higher

You must have one of these environments or their associated Java runtime environments to use the JDBC Driver. For details on obtaining this software, see the URL <http://java.sun.com/products/index.html>.

### 3.3 SETTING UP THE JDBC DRIVER: WEB SERVER

In a Web server environment, the JDBC driver and Java applets that use it reside on a host system. No additional driver software is required on the client machine. Client applications must support a Java virtual machine compatible with JavaSoft's JDK Version 1.4 or later. (Internet browsers such as Netscape or Internet Explorer meet this requirement.)

Client applications invoke a JDBC applet through a Web page on the server. The browser downloads both the applet and the JDBC Driver from the server (usually in compressed format) and runs the applet. The Java applet opens a database connection (see [Section 3.5 "Connecting to a Database" on page 3-4](#)) and accesses the database using the JDBC API.

A general JDBC security restriction is that applets can only open a database connection from the server from which they are downloaded. That means the host system must be running both the HTTP Web server and the c-treeSQL Server - Java Edition process.

To set up the JDBC Driver for an applet on a Web server, complete these steps:

- Copy compiled class files for the JDBC Driver and the applet to a directory accessible to the Web page that will invoke the applet.
- Compress all the class files into a single Java Archive (JAR) file.
- Create the Web page that will invoke the applet.

The following sections describe these steps.

### 3.3.1 Copying JDBC Driver and Applet Class Files

On both Windows NT/2000 and UNIX, the class files for the JDBC Driver are created in the *classes\ctreeJDBC.jar* file. Copy the jar files from that directory to a directory accessible to the Web page. Do the same for the applet's class file.

For example:

```
systpe@isis% cd $webroot
/vol6/webroot
systpe@isis% mkdir test
systpe@isis% cd test
/vol6/webroot/test
systpe@isis% cp -i $TPEROOT/classes/ctreeJDBC.jar
systpe@isis% cp -i /applet_test/DhJDBCApplet.class
```

### 3.3.2 Creating a Web Page That Invokes the Applet

At a minimum, the page must include the APPLET tag that invokes the applet.

For example, the following page includes little else but the APPLET tag. The example's APPLET tag specifies the *DhJDBCApplet.class* and *DhJDBCTest.jar* files from the preceding sections, as well as class-name and connection parameters to pass to the applet.

```
ADMIN@isis% pwd
/vol6/webroot/test
ADMIN@isis% more jtest.htm
<html>
    <head>
        <title>Test</title>
    </head>
    <body>
        <p>
            Here, in all its glory, is the DhJDBCApplet test applet!
        <center>
            <applet code="DhJDBCApplet.class"
                archive="DhJDBCTest.jar" width=500 height=400>
                <param name=Driver value="ctree.jdbc.ctreeDriver">
                <param name=URL value="jdbc:ctree:6597@isis:jdbcdb">
                <param name=User value="ADMIN">
                <param name=Password value="dummy">
            </applet>
        </center>
    </body>
</html>
```

## 3.4 SETTING UP THE JDBC DRIVER: APPLICATION SERVER

In an application server environment, the system on which the JDBC application runs also has the JDBC driver installed. This configuration provides good performance when users are on the same system or can execute the JDBC application across a network.

To set up the JDBC Driver, you must have access to a system (UNIX or NT) where the c-treeSQL libraries and executable files have been built. On both Windows NT and UNIX, the JDBC class files are created in the *classes\ctreeJDBC.jar* file.

If the application server is a different system than the system used for those c-treeSQL libraries and executables, you may need to copy the jar file to the application server. Then, you need to set the CLASSPATH environment variable to include the directory where the class files reside.

The following sections describe these steps.

### 3.4.1 Copying JDBC Driver Files to the Application Server (If Necessary)

This step is not necessary if the application server has access to the disk containing the class files (through a Windows network or NFS-mounted disks on UNIX).

Otherwise, you must first copy files to the application server. On UNIX systems and Windows systems, use any available utility to copy the jar files to the application server.

### 3.4.2 Setting Environment Variables

Whether the JDBC Driver class files reside locally or on network-served disks, you must set the CLASSPATH environment variable to point to the class files.

On both Windows NT and UNIX, the CLASSPATH environment variable must point to the jar file *ctreeJDBC.jar*.

On Windows NT, you can set the CLASSPATH environment variable at the Windows NT command prompt, as in the following example:

```
set CLASSPATH=%CLASSPATH%;
.;
c:\jdk1.3.1_04\jre\lib\rt.jar;c:\FairCom\c-treeServers\
  c-treeJQLServer\classes\ctreeJDBC.jar
```

(For environment variables to persist across different processes, set them using the Window Control Panel's *System* utility, and set them as system variables.)

Also, make sure the PATH environment variable includes the appropriate Java compiler directory. In Windows NT, the syntax would be as follows:

```
PATH=%PATH%;
  c:\jdk1.4.2_01\bin;
  c:\jdk1.4.2_01\jre\bin\classic
```

(Again, for the environment variables to persist across different processes, set them using the Window Control Panel's *System* utility, and set them as system variables.)

### 3.4.3 Running the Sample Application

The c-treeSQL distribution includes a test Java application, *ctreeTest.class*, that exercises the JDBC interfaces implemented by the JDBC Driver. The application connects to the specified database (or creates a default connection hardcoded in the application) and presents a limited interactive SQL interface.

You can run the sample application to check the setup of the JDBC Driver in an application server environment.

For example, on Windows NT (using the JavaSoft JDK), the following command line supplies a connection URL to run the sample program (see [Section 3.5 "Connecting to a Database" on page 3-4](#) for details on specifying a URL):

```
C:\>java ctreeTest jdbc:ctree:6597@ktwo:kirk2
```

If you have a Java compiler, you can copy the sample program source code and modify it. The source file is *src\jdbc\ctreeTest.java* on the system where the class files were built.

## 3.5 CONNECTING TO A DATABASE

JDBC applications must perform two steps to connect to a database:

1. Load the JDBC driver
2. Connect to the driver

### 3.5.1 Load the JDBC Driver Using *Class.forName*

The *Class.forName* method takes as its argument the fully-qualified class name for the JDBC Driver. If it finds the class, the method loads and links the class, and returns the Class object representing the class.

The fully-qualified class name for the c-treeSQL JDBC Driver is *ctree.jdbc.ctreeDriver*. To load the JDBC Driver, use it as the argument to the *Class.forName* method:

```
// Load the driver
Class.forName ("ctree.jdbc.ctreeDriver");
```

### 3.5.2 Connect to the JDBC Driver Using *DriverManager.getConnection*

To connect to a c-treeSQL database through the JDBC Driver, an application specifies:

- A database connection string in the form of a JDBC URL
- User authentication detail (user name and password)

Applications specify this information as arguments to the *DriverManager.getConnection* method.



3.5.2.1 Java URL Connection String

*DriverManager.getConnection* requires at least one argument, a character string specifying a database connection URL. For the c-treeSQL JDBC Driver, the URL takes the following form:

```
jdbc:ctree:port@host_name:db_name
```

The URL string has the following components:

|                   |                                                                               |
|-------------------|-------------------------------------------------------------------------------|
| <b>jdbc:ctree</b> | An identifying protocol and subprotocol string for the c-treeSQL JDBC Driver. |
| <b>:port</b>      | The port number associated with the JDBC server on the host system.           |
| <b>@host_name</b> | Name of the server system where the database resides.                         |
| <b>:db_name</b>   | Name of the database.                                                         |

For example, the default URL in the sample application is *jdbc:ctree:6597@isis:testdb*. When passed to *DriverManager.getConnection*, this URL specifies that the c-treeSQL JDBC Driver be used to connect to the database *testdb* on the server named *isis*.

3.5.2.2 User Authentication Detail

*DriverManager.getConnection* accepts three variants of user authentication detail:

- User name and password passed as two character string arguments:

```
Connection con = DriverManager.getConnection (url, "fred" "fredpasswd" );
```

- User name and password passed as a single *Properties* object:

```
Connection con = DriverManager.getConnection (url, prop );
```

Note that the JDBC Driver expects the keys of the *Properties* object to be named user and password when it processes the object. Application code must use those names when it populates the *Properties* object:

```
prop.put("user", userid);  
prop.put("password", passwd);
```

- User name and password omitted. The JDBC Driver connects to the database with a blank username and null password:

```
Connection con = DriverManager.getConnection (url);
```

3.5.3 An Example Connection

The following example shows a code excerpt that illustrates loading the driver and connecting to the default server and database. The following example uses the form of *DriverManager.getConnection* that takes authentication information as a single *Properties* object.

**Example 3-1: Loading the JDBC Driver and Connecting to a Database**

```
String url    = "jdbc:ctree:6597@isis:testdb";
```

```
String userid = "fred";
String passwd = "fredpasswd";

// Load the driver
Class.forName ("ctree.jdbc.ctreeDriver");

// Attempt to connect to a driver.  Each one
// of the registered drivers will be loaded until
// one is found that can process this URL.
java.util.Properties prop = new java.util.Properties();
prop.put("user", userid);
prop.put("password", passwd);

Connection con = DriverManager.getConnection (url, prop);
```

### 3.5.4 Connection Pooling Support in JDBC

The IBM's Websphere 4.0 Application server uses the JDBC APIs `DataSource`, `PooledConnection` and `ConnectionPoolDataSource` to connect to any data source. These APIs are part of the `javax.sql` package. It is mandatory to support them when working with IBM's Websphere 4.0 middleware.

#### 3.5.4.1 `javax.sql` Package

The `javax.sql` package provides the APIs for server side data source access and processing. This is included in the Java 2 SDK version 1.4, Standard Edition.

The `javax.sql` package provides the following:

- The `DataSource` interface as an alternative to the `DriverManager` for establishing a connection with a data source
- Connection pooling

#### 3.5.4.2 Using A `DataSource` Object To Make A Connection

The `javax.sql` package provides the preferred way to make a connection with a data source. The `DriverManager` class, the original mechanism, is still valid, and code using it will continue to run. However, the newer `DataSource` mechanism is preferred because it offers many advantages over the `DriverManager` mechanism.

The following are the main advantages of using a `DataSource` object to make a connection:

- Applications do not need to hard code a driver class.
- Changes can be made to a data source's properties, which means that it is not necessary to make changes in application code when something about the data source or driver changes.
- Connection pooling and distributed transactions are available through a `DataSource` object that is implemented to work with the middle-tier infrastructure. Connections made through the `DriverManager` do not have connection pooling or distributed transaction capabilities.

A particular `DataSource` object represents a particular physical data source, and each connection the `DataSource` object creates is a connection to that physical data source. A `DataSource` object can be implemented to work with the middle tier infrastructure so that the connections it produces will be pooled for reuse. An application that uses such a `DataSource` implementation will automatically get a connection that participates in connection pooling. A `DataSource` object can also be implemented to work with the middle tier infrastructure so that the connections it produces can be used for distributed transactions without any special coding.

### **3.5.4.3 Connection Pooling**

Connections made via a `DataSource` object that is implemented to work with a middle tier connection pool manager will participate in connection pooling. This can improve performance dramatically because creating new connections is very expensive. Connection pooling allows a connection to be used and reused, thus reducing the number of new connections that need to be created.

Connection pooling is totally transparent. It is done automatically in the middle tier of a J2EE Environment, and hence from an application's viewpoint, no change in code is required. An application simply uses the `DataSource.getConnection` method to get the pooled connection and uses it the same way it uses any `Connection` object.

### **3.5.4.4 Implementation in Dharma JDBC Driver**

In order to support working with IBM's Websphere 4.0 Application server, the following interfaces from the package `javax.sql` have been implemented.

- `javax.sql.DataSource`

This is a factory for connections to the physical data source that this `DataSource` object represents. An object that implements the `DataSource` interface will typically be registered with a naming service based on the Java Naming and Directory (JNDI) API.

- `javax.sql.PooledConnection` interfaces

An object that provides hooks for connection pool management. A `PooledConnection` object represents a physical connection to a data source. The connection can be recycled rather than being closed when an application is finished with it, thus reducing the number of connections that need to be made.

- `javax.sql.ConnectionPoolDataSource`

This is a factory for `PooledConnection` objects. An object that implements this interface will typically be registered with a naming service that is based on the Java Naming and Directory Interface (JNDI).

A new directory, `TPEROOT/src/jdbcx`, has been introduced. This directory has the following source files:

- `DharmaDataSource.java`

- DharmaConnectionPoolDataSource.java
- DharmaPooledConnection.java
- DharmaObjectFactory.java

### 3.6 MANAGING TRANSACTIONS EXPLICITLY TO IMPROVE PERFORMANCE

By default, new connections in JDBC applications are in autocommit mode.

In autocommit mode every SQL statement executes in its own transaction:

- After successful completion, the JDBC Driver automatically commits the transaction.
- If the statement execution fails, the JDBC Driver automatically rolls back the transaction.

**Note:** In autocommit mode, the JDBC Driver does not issue a commit after SELECT and CALL statements. The driver assumes these statements generate result sets and relies on the application to explicitly commit or roll back the transaction after it processes any result set and closes the statement.

You can change transaction mode to manual commit by calling the *Connection.setAutoCommit* method. In manual commit mode, applications must commit a transaction by using the *Connection.commit* method. Similarly, applications must explicitly roll back a transaction by invoking the *Connection.rollback* method.

You will improve the performance of your programs by setting autocommit to false after creating a Connection object with the *Connection.setAutoCommit* method:

```
Connection con = DriverManager.getConnection ( url, prop);  
.  
.  
.  
con.setAutoCommit(false);
```

# JDBC Conformance Notes

## 4.1 SUPPORTED DATA TYPES

The c-treeSQL ODBC Driver supports standard JDBC mapping of JDBC types to corresponding Java types.

In the JDBC methods *CallableStatement.getXXX* and *PreparedStatement.setXXX* methods, *XXX* is a Java type:

- For *setXXX* methods, the driver converts the Java type to the JDBC type shown in the following table before sending it to the database.
- For *getXXX* methods, the driver converts the JDBC type returned by the database to the Java type shown in [Table 4-2: Mapping Between JDBC and Java Data Types on page 4-1](#) before returning it to the *getXXX* method.

**Table 4-1: Mapping Between Java and JDBC Data Types**

| Java Type            | JDBC type              | Java Type          | JDBC type                  |
|----------------------|------------------------|--------------------|----------------------------|
| String               | VARCHAR or LONGVARCHAR | float              | REAL                       |
| java.math.BigDecimal | NUMERIC                | double             | DOUBLE                     |
| boolean              | BIT                    | byte[]             | VARBINARY or LONGVARBINARY |
| byte                 | TINYINT                | java.sql.Date      | DATE                       |
| short                | SMALLINT               | java.sql.Time      | TIME                       |
| int                  | INTEGER                | java.sql.Timestamp | TIMESTAMP                  |
| long                 | BIGINT                 |                    |                            |

**Table 4-2: Mapping Between JDBC and Java Data Types**

| JDBC type | Java type | JDBC type | Java type |
|-----------|-----------|-----------|-----------|
| CHAR      | String    | REAL      | float     |

Table 4-2: Mapping Between JDBC and Java Data Types

| JDBC type   | Java type            | JDBC type     | Java type          |
|-------------|----------------------|---------------|--------------------|
| VARCHAR     | String               | FLOAT         | double             |
| LONGVARCHAR | String               | DOUBLE        | double             |
| NUMERIC     | java.math.BigDecimal | BINARY        | byte[]             |
| DECIMAL     | java.math.BigDecimal | VARBINARY     | byte[]             |
| BIT         | boolean              | LONGVARBINARY | byte[]             |
| TINYINT     | byte                 | DATE          | java.sql.Date      |
| SMALLINT    | short                | TIME          | java.sql.Time      |
| INTEGER     | int                  | TIMESTAMP     | java.sql.Timestamp |
| BIGINT      | long                 |               |                    |

## 4.2 RETURN VALUES FOR *DATABASEMETADATA* METHODS

Applications call methods of the *DatabaseMetaData* class to retrieve details about the JDBC support provided by a specific driver.

The following table lists each method of the *DatabaseMetadata* class and shows what the c-treeSQL JDBC Driver returns when an applications calls the method. For details on the format and usage of each method, see the Java Platform Core API documentation.

The following example shows an excerpt from the sample program that illustrates calling methods of *DatabaseMetadata*.

### Example 4-1: Getting Driver Information Through DatabaseMetadata Methods

```
Connection con = DriverManager.getConnection ( url, prop);
```

```
.  
.
.
```

```
    // Get the DatabaseMetaData object and display  
    // some information about the connection
```

```
DatabaseMetaData dma = con.getMetaData ();
```

```
o.println("\nConnected to " + dma.getURL());  
o.println("Driver      " + dma.getDriverName());  
o.println("Version      " + dma.getDriverVersion());
```

Many of the methods return lists of information as an object of type *ResultSet*. Use the normal *ResultSet* methods such as *getString* and *getInt* to retrieve the data from the result sets.

**Table 4-3: Return Values for *DatabaseMetaData* Methods**

| Method                                                                  | Description                                                                                  | Returns              |
|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|----------------------|
| <code>allProceduresAreCallable()</code>                                 | Can all the procedures returned by <code>getProcedures</code> be called by the current user? | True                 |
| <code>allTablesAreSelectable()</code>                                   | Can all the tables returned by <code>getTable</code> be SELECTed by the current user?        | False                |
| <code>dataDefinitionCausesTransactionCommit()</code>                    | Does a data definition statement within a transaction force the transaction to commit?       | False                |
| <code>dataDefinitionIgnoredInTransactions ()</code>                     | Is a data definition statement within a transaction ignored?                                 | False                |
| <code>doesMaxRowSizeIncludeBlobs()</code>                               | Did <code>getMaxRowSize()</code> include LONGVARCHAR and LONG-VARBINARY blobs?               | False                |
| <code>getBestRowIdentifier(String, String, String, int, boolean)</code> | Get a description of a table's optimal set of columns that uniquely identifies a row.        | (result set)         |
| <code>getCatalogs()</code>                                              | Get the catalog names available in this database.                                            | "Driver not capable" |
| <code>getCatalogSeparator()</code>                                      | What's the separator between catalog and table name?                                         | "" (blank)           |
| <code>getCatalogTerm()</code>                                           | What's the database vendor's preferred term for "catalog"?                                   | "" (blank)           |
| <code>getColumnPrivileges(String, String, String, String)</code>        | Get a description of the access rights for a table's columns.                                | (result set)         |
| <code>getColumns(String, String, String, String)</code>                 | Get a description of table columns available in a catalog.                                   | (result set)         |

Table 4-3: Return Values for *DatabaseMetaData* Methods

| Method                                                            | Description                                                                                                                                                                                                                                                                                                                                                                                   | Returns                  |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| getCrossReference(String, String, String, String, String, String) | Get a description of the foreign key columns in the foreign key table that reference the primary key columns of the primary key table (describe how one table imports another's key.) This should normally return a single foreign key/primary key pair (most tables only import a foreign key from a table once.) They are ordered by FKTABLE_CAT, FKTABLE_SCHEM, FKTABLE_NAME, and KEY_SEQ. | (result set)             |
| getDatabaseProductName()                                          | What's the name of this database product?                                                                                                                                                                                                                                                                                                                                                     | "c-treeSQL"              |
| getDatabaseProductVersion()                                       | What's the version of this database product?                                                                                                                                                                                                                                                                                                                                                  | "09.00.0000"             |
| getDefaultTransactionIsolation()                                  | What's the database's default transaction isolation level? The values are defined in java.sql.Connection.                                                                                                                                                                                                                                                                                     | TRANSACTION_SERIALIZABLE |
| getDriverMajorVersion()                                           | What's this JDBC driver's major version number?                                                                                                                                                                                                                                                                                                                                               | 9.0                      |
| getDriverMinorVersion()                                           | What's this JDBC driver's minor version number?                                                                                                                                                                                                                                                                                                                                               | 0                        |
| getDriverName()                                                   | What's the name of this JDBC driver?                                                                                                                                                                                                                                                                                                                                                          | "ctree.jdbc.ctreeDriver" |
| getDriverVersion()                                                | What's the version of this JDBC driver?                                                                                                                                                                                                                                                                                                                                                       | "09.00.0000"             |
| getExportedKeys(String, String, String)                           | Get a description of the foreign key columns that reference a table's primary key columns (the foreign keys exported by a table).                                                                                                                                                                                                                                                             | (result set)             |
| getExtraNameCharacters()                                          | Get all the "extra" characters that can be used in unquoted identifier names (those beyond a-z, A-Z, 0-9 and _).                                                                                                                                                                                                                                                                              | null                     |



Table 4-3: Return Values for *DatabaseMetaData* Methods

| Method                                                 | Description                                                                                                                               | Returns      |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| getIdentifierQuoteString ()                            | What's the string used to quote SQL identifiers? This returns a space " " if identifier quoting isn't supported.                          | ""           |
| getImportedKeys(String, String, String)                | Get a description of the primary key columns that are referenced by a table's foreign key columns (the primary keys imported by a table). | (result set) |
| getIndexInfo(String, String, String, boolean, boolean) | Get a description of a table's indices and statistics.                                                                                    | (result set) |
| getMaxBinaryLiteralLength()                            | How many hex characters can you have in an inline binary literal?                                                                         | 32000        |
| getMaxCatalogNameLength()                              | What's the maximum length of a catalog name?                                                                                              | 32           |
| getMaxCharLiteralLength()                              | What's the max length for a character literal?                                                                                            | 2000         |
| getMaxColumnNameLength()                               | What's the limit on column name length?                                                                                                   | 32           |
| getMaxColumnsInGroupBy()                               | What's the maximum number of columns in a "GROUP BY" clause?                                                                              | 0 (no limit) |
| getMaxColumnsInIndex()                                 | What's the maximum number of columns allowed in an index?                                                                                 | 100          |
| getMaxColumnsInOrderBy()                               | What's the maximum number of columns in an "ORDER BY" clause?                                                                             | 0 (no limit) |
| getMaxColumnsInSelect()                                | What's the maximum number of columns in a "SELECT" list?                                                                                  | 0 (no limit) |
| getMaxColumnsInTable()                                 | What's the maximum number of columns in a table?                                                                                          | 500          |
| getMaxConnections()                                    | How many active connections can we have at a time to this database?                                                                       | 10           |

**Table 4-3: Return Values for *DatabaseMetaData* Methods**

| Method                      | Description                                                               | Returns      |
|-----------------------------|---------------------------------------------------------------------------|--------------|
| getMaxCursorNameLength()    | What's the maximum cursor name length?                                    | 32           |
| getMaxIndexLength()         | What's the maximum length of an index (in bytes)?                         | 0 (no limit) |
| getMaxProcedureNameLength() | What's the maximum length of a procedure name?                            | 32           |
| getMaxRowSize()             | What's the maximum length of a single row?                                | 0 (no limit) |
| getMaxSchemaNameLength()    | What's the maximum length allowed for a schema name?                      | 32           |
| getMaxStatementLength()     | What's the maximum length of a SQL statement?                             | 35000        |
| getMaxStatements()          | How many active statements can we have open at one time to this database? | 0 (no limit) |
| getMaxTableNameLength()     | What's the maximum length of a table name?                                | 32           |
| getMaxTablesInSelect()      | What's the maximum number of tables in a SELECT?                          | 250          |
| getMaxUserNameLength()      | What's the maximum length of a user name?                                 | 32           |

Table 4-3: Return Values for *DatabaseMetaData* Methods

| Method                                                           | Description                                                                      | Returns                                                                                                                                                                                                                     |
|------------------------------------------------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>getNumericFunctions()</code>                               | Get a comma separated list of math functions.                                    | ABS,<br>ACOS,<br>ASIN,<br>ATAN,<br>ATAN2,<br>CEILING,<br>COS,<br>COT,<br>DEGREES,<br>EXP,<br>FLOOR,<br>LOG,<br>LOG10,<br>MOD,<br>PI,<br>POWER,<br>RADIANS,<br>RAND,<br>ROUND,<br>SIGN,<br>SIN,<br>SQRT,<br>TAN,<br>TRUNCATE |
| <code>getPrimaryKeys(String, String, String)</code>              | Get a description of a table's primary key columns.                              | (result set)                                                                                                                                                                                                                |
| <code>getProcedureColumns(String, String, String, String)</code> | Get a description of a catalog's stored procedure parameters and result columns. | (result set)                                                                                                                                                                                                                |
| <code>getProcedures(String, String, String)</code>               | Get a description of stored procedures available in a catalog.                   | (result set)                                                                                                                                                                                                                |
| <code>getProcedureTerm()</code>                                  | What's the database vendor's preferred term for "procedure"?                     | "procedure"                                                                                                                                                                                                                 |
| <code>getSchemas()</code>                                        | Get the schema names available in this database.                                 | (result set)                                                                                                                                                                                                                |
| <code>getSchemaTerm()</code>                                     | What's the database vendor's preferred term for "schema"?                        | "Owner"                                                                                                                                                                                                                     |

**Table 4-3: Return Values for *DatabaseMetaData* Methods**

| Method                                       | Description                                                                                                     | Returns                                                                                                                                                                               |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| getSearchStringEscape()                      | This is the string that can be used to escape '_' or '%' in the string pattern style catalog search parameters. | "\"                                                                                                                                                                                   |
| getSQLKeywords()                             | Get a comma separated list of all a database's SQL keywords that are NOT also SQL92 keywords.                   | null                                                                                                                                                                                  |
| getStringFunctions()                         | Get a comma separated list of string functions.                                                                 | ASCII,<br>LTRIM,<br>CHAR,<br>DIFFERENCE,<br>INSERT,<br>LCASE,<br>LEFT,<br>REPEAT,<br>REPLACE,<br>SOUNDEX,<br>SPACE,<br>SUBSTRING,<br>UCASE,<br>RTRIM,<br>CONCAT,<br>LENGTH,<br>LOCATE |
| getSystemFunctions()                         | Get a comma separated list of system functions.                                                                 | SQL_FN_SYS_USERN<br>AME,<br>SQL_FN_SYS_IFNULL<br>,<br>SQL_FN_SYS_DBNAM<br>E                                                                                                           |
| getTablePrivileges(String, String, String)   | Get a description of the access rights for each table available in a catalog.                                   | (result set)                                                                                                                                                                          |
| getTables(String, String, String, String []) | Get a description of tables available in a catalog.                                                             | (result set)                                                                                                                                                                          |
| getTableTypes()                              | Get the table types available in this database.                                                                 | SYNONYM, SYSTEM<br>TABLE, TABLE, VIEW                                                                                                                                                 |

Table 4-3: Return Values for *DatabaseMetaData* Methods

| Method                                    | Description                                                                                               | Returns                                                                                                                                   |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| getTimeDateFunctions()                    | Get a comma separated list of time and date functions.                                                    | CURDATE,<br>CURTIME,<br>DAYOFMONTH,<br>DAYOFWEEK,<br>DAYOFYEAR,<br>MONTHNOW,<br>QUARTER,<br>WEEK,<br>YEAR,<br>HOUR,<br>MINUTE,<br>SECOND, |
| getTypeInfo()                             | Get a description of all the standard SQL types supported by this database.                               | (result set)                                                                                                                              |
| getURL()                                  | What's the url for this database?                                                                         | (the URL)                                                                                                                                 |
| getUserName()                             | What's our user name as known to the database?                                                            | (the userid)                                                                                                                              |
| getVersionColumns(String, String, String) | Get a description of a table's columns that are automatically updated when any value in a row is updated. | (result set)                                                                                                                              |
| isCatalogAtStart()                        | Does a catalog appear at the start of a qualified table name? (Otherwise it appears at the end)           | True                                                                                                                                      |
| isReadOnly()                              | Is the database in read-only mode?                                                                        | False                                                                                                                                     |
| nullPlusNonNullsNull()                    | Are concatenations between NULL and non-NULL values NULL? A JDBC-Compliant driver always returns true.    | True                                                                                                                                      |
| nullsAreSortedAtEnd()                     | Are NULL values sorted at the end regardless of sort order?                                               | False                                                                                                                                     |
| nullsAreSortedAtStart()                   | Are NULL values sorted at the start regardless of sort order?                                             | False                                                                                                                                     |
| nullsAreSortedHigh()                      | Are NULL values sorted high?                                                                              | False                                                                                                                                     |
| nullsAreSortedLow()                       | Are NULL values sorted low?                                                                               | True                                                                                                                                      |

**Table 4-3: Return Values for *DatabaseMetaData* Methods**

| Method                             | Description                                                                                                   | Returns                                                                                    |
|------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| storesLowerCaseIdentifiers()       | Does the database treat mixed case unquoted SQL identifiers as case insensitive and store them in lower case? | True or False<br>Depends on the identifier case specified during the creation of database. |
| storesLowerCaseQuotedIdentifiers() | Does the database treat mixed case quoted SQL identifiers as case insensitive and store them in lower case?   | True or False<br>Depends on the identifier case specified during the creation of database. |
| storesMixedCaseIdentifiers()       | Does the database treat mixed case unquoted SQL identifiers as case insensitive and store them in mixed case? | False                                                                                      |
| storesMixedCaseQuotedIdentifiers() | Does the database treat mixed case quoted SQL identifiers as case insensitive and store them in mixed case?   | False                                                                                      |
| storesUpperCaseIdentifiers()       | Does the database treat mixed case unquoted SQL identifiers as case insensitive and store them in upper case? | True or False<br>Depends on the identifier case specified during the creation of database. |
| storesUpperCaseQuotedIdentifiers() | Does the database treat mixed case quoted SQL identifiers as case insensitive and store them in upper case?   | False                                                                                      |
| supportsAlterTableWithAddColumn()  | Is "ALTER TABLE" with add column supported?                                                                   | True                                                                                       |
| supportsAlterTableWithDropColumn() | Is "ALTER TABLE" with drop column supported?                                                                  | True                                                                                       |
| supportsANSI92EntryLevelSQL()      | Is the ANSI92 entry level SQL grammar supported? All JDBC-Compliant drivers must return true.                 | True                                                                                       |
| supportsANSI92FullSQL()            | Is the ANSI92 full SQL grammar supported?                                                                     | False                                                                                      |

**Table 4-3: Return Values for *DatabaseMetaData* Methods**

| Method                                                               | Description                                                                                                                                                 | Returns                                               |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| <code>supportsANSI92IntermediateSQL()</code>                         | Is the ANSI92 intermediate SQL grammar supported?                                                                                                           | False                                                 |
| <code>supportsCatalogsInDataManipulation()</code>                    | Can a catalog name be used in a data manipulation statement?                                                                                                | False                                                 |
| <code>supportsCatalogsInIndexDefinitions()</code>                    | Can a catalog name be used in an index definition statement?                                                                                                | False                                                 |
| <code>supportsCatalogsInPrivilegeDefinitions()</code>                | Can a catalog name be used in a privilege definition statement?                                                                                             | False                                                 |
| <code>supportsCatalogsInProcedureCalls()</code>                      | Can a catalog name be used in a procedure call statement?                                                                                                   | False                                                 |
| <code>supportsCatalogsInTableDefinitions()</code>                    | Can a catalog name be used in a table definition statement?                                                                                                 | False                                                 |
| <code>supportsColumnAliasing()</code>                                | Is column aliasing supported? If so, the SQL AS clause can be used to provide names for computed columns or to provide alias names for columns as required. | True                                                  |
| <code>supportsConvert()</code>                                       | Is the CONVERT function between SQL types supported?                                                                                                        | True                                                  |
| <code>supportsConvert(int, int)</code>                               | Is CONVERT between the given SQL types supported?                                                                                                           | True or False<br>Depends on the types being converted |
| <code>supportsCoreSQLGrammar()</code>                                | Is the ODBC Core SQL grammar supported?                                                                                                                     | True                                                  |
| <code>supportsCorrelatedSubqueries()</code>                          | Are correlated subqueries supported? A JDBC-Compliant driver always returns true.                                                                           | True                                                  |
| <code>supportsDataDefinitionAndDataManipulationTransactions()</code> | Are both data definition and data manipulation statements within a transaction supported?                                                                   | True                                                  |
| <code>supportsDataManipulationTransactionsOnly()</code>              | Are only data manipulation statements within a transaction supported?                                                                                       | False                                                 |

**Table 4-3: Return Values for *DatabaseMetaData* Methods**

| Method                                   | Description                                                                                                                                                               | Returns |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| supportsDifferentTableCorrelationNames() | If table correlation names are supported, are they restricted to be different from the names of the tables?                                                               | False   |
| supportsExpressionsInOrderBy()           | Are expressions in "ORDER BY" lists supported?                                                                                                                            | True    |
| supportsExtendedSQLGrammar()             | Is the ODBC Extended SQL grammar supported?                                                                                                                               | True    |
| supportsFullOuterJoins()                 | Are full nested outer joins supported?                                                                                                                                    | False   |
| supportsGroupBy()                        | Is some form of "GROUP BY" clause supported?                                                                                                                              | True    |
| supportsGroupByBeyondSelect()            | Can a "GROUP BY" clause add columns not in the SELECT provided it specifies all the columns in the SELECT?                                                                | True    |
| supportsGroupByUnrelated()               | Can a "GROUP BY" clause use columns not in the SELECT?                                                                                                                    | False   |
| supportsIntegrityEnhancementFacility()   | Is the SQL Integrity Enhancement Facility supported?                                                                                                                      | True    |
| supportsLikeEscapeClause()               | Is the escape character in "LIKE" clauses supported? A JDBC-Compliant driver always returns true.                                                                         | True    |
| supportsLimitedOuterJoins()              | Is there limited support for outer joins? (This will be true if supportFullOuterJoins is true.)                                                                           | True    |
| supportsMinimumSQLGrammar()              | Is the ODBC Minimum SQL grammar supported? All JDBC-Compliant drivers must return true.                                                                                   | True    |
| supportsMixedCaseIdentifiers()           | Does the database treat mixed case unquoted SQL identifiers as case sensitive and as a result store them in mixed case? A JDBC-Compliant driver will always return false. | False   |



**Table 4-3: Return Values for *DatabaseMetaData* Methods**

| Method                                 | Description                                                                                                                                                            | Returns |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| supportsMixedCaseQuotedIdentifiers()   | Does the database treat mixed case quoted SQL identifiers as case sensitive and as a result store them in mixed case? A JDBC-Compliant driver will always return true. | True    |
| supportsMultipleResultSets()           | Are multiple ResultSets from a single execute supported?                                                                                                               | False   |
| supportsMultipleTransactions ()        | Can we have multiple transactions open at once (on different connections)?                                                                                             | True    |
| supportsNonNullableColumns()           | Can columns be defined as non-nullable? A JDBC-Compliant driver always returns true.                                                                                   | True    |
| supportsOpenCursorsAcrossCommit()      | Can cursors remain open across commits?                                                                                                                                | False   |
| supportsOpenCursorsAcrossRollback()    | Can cursors remain open across rollbacks?                                                                                                                              | False   |
| supportsOpenStatementsAcrossCommit()   | Can statements remain open across commits?                                                                                                                             | True    |
| supportsOpenStatementsAcrossRollback() | Can statements remain open across rollbacks?                                                                                                                           | True    |
| supportsOrderByUnrelated()             | Can an "ORDER BY" clause use columns not in the SELECT?                                                                                                                | True    |
| supportsOuterJoins()                   | Is some form of outer join supported?                                                                                                                                  | True    |
| supportsPositionedDelete()             | Is positioned DELETE supported?                                                                                                                                        | True    |
| supportsPositionedUpdate()             | Is positioned UPDATE supported?                                                                                                                                        | True    |
| supportsSchemasInDataManipulation()    | Can a schema name be used in a data manipulation statement?                                                                                                            | True    |
| supportsSchemasInIndexDefinitions()    | Can a schema name be used in an index definition statement?                                                                                                            | True    |

**Table 4-3: Return Values for *DatabaseMetaData* Methods**

| Method                                  | Description                                                                                        | Returns |
|-----------------------------------------|----------------------------------------------------------------------------------------------------|---------|
| supportsSchemasInPrivilegeDefinitions() | Can a schema name be used in a privilege definition statement?                                     | True    |
| supportsSchemasInProcedureCalls()       | Can a schema name be used in a procedure call statement?                                           | True    |
| supportsSchemasInTableDefinitions()     | Can a schema name be used in a table definition statement?                                         | True    |
| supportsSelectForUpdate()               | Is SELECT for UPDATE supported?                                                                    | True    |
| supportsStoredProcedures()              | Are stored procedure calls using the stored procedure escape syntax supported?                     | True    |
| supportsSubqueriesInComparisons()       | Are subqueries in comparison expressions supported? A JDBC-Compliant driver always returns true.   | True    |
| supportsSubqueriesInExists()            | Are subqueries in 'exists' expressions supported? A JDBC-Compliant driver always returns true.     | True    |
| supportsSubqueriesInIns()               | Are subqueries in 'in' statements supported? A JDBC-Compliant driver always returns true.          | True    |
| supportsSubqueriesInQuantifieds()       | Are subqueries in quantified expressions supported? A JDBC-Compliant driver always returns true.   | True    |
| supportsTableCorrelationNames()         | Are table correlation names supported? A JDBC-Compliant driver always returns true.                | True    |
| supportsTransactionIsolationLevel(int)  | Does the database support the given transaction isolation level?                                   | True    |
| supportsTransactions ()                 | Are transactions supported? If not, commit is a no-op and the isolation level is TRANSACTION_NONE. | True    |
| supportsUnion()                         | Is SQL UNION supported?                                                                            | True    |

Table 4-3: Return Values for *DatabaseMetaData* Methods

| Method                                              | Description                                                                                                     | Returns                                              |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <code>supportsUnionAll()</code>                     | Is SQL UNION ALL supported?                                                                                     | True                                                 |
| <code>usesLocalFilePerTable()</code>                | Does the database use a file for each table?                                                                    | False                                                |
| <code>usesLocalFiles()</code>                       | Does the database store tables in a local file?                                                                 | False                                                |
| <b>JDBC 2.0</b>                                     |                                                                                                                 |                                                      |
| <code>deletesAreDetected(int)</code>                | Indicates whether or not a visible row delete can be detected by calling <code>ResultSet.rowDeleted()</code> .  | False                                                |
| <code>getConnection()</code>                        | Retrieves the connection that produced this metadata object.                                                    | The connection that produced this metadata object    |
| <code>getUDTs(String, String, String, int[])</code> | Gets a description of the user-defined types defined in a particular schema.                                    | Empty <code>ResultSet</code> object                  |
| <code>insertsAreDetected(int)</code>                | Indicates whether or not a visible row insert can be detected by calling <code>ResultSet.rowInserted()</code> . | False                                                |
| <code>othersDeletesAreVisible(int)</code>           | Indicates whether deletes made by others are visible.                                                           | False                                                |
| <code>othersUpdatesAreVisible(int)</code>           | Indicates whether updates made by others are visible.                                                           | False                                                |
| <code>ownDeletesAreVisible(int)</code>              | Indicates whether a result set's own deletes are visible.                                                       | False                                                |
| <code>ownInsertsAreVisible(int)</code>              | Indicates whether inserts made by others are visible.                                                           | False                                                |
| <code>ownUpdatesAreVisible(int)</code>              | Indicates whether a result set's own updates are visible.                                                       | False                                                |
| <code>supportsBatchUpdates()</code>                 | Indicates whether the driver supports batch updates.                                                            | True                                                 |
| <code>supportsResultSetType(int)</code>             | Does the database support the given result set type?                                                            | True if result set type is <code>FORWARD_ONLY</code> |

Table 4-3: Return Values for *DatabaseMetaData* Methods

| Method                                                             | Description                                                                                                                                           | Returns                                                                                                       |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>supportsResultSetConcurrency(int, int)</code>                | Does the database support the concurrency type in combination with the given result set type?                                                         | True if result set type is <code>FORWARD_ONLY</code> and if concurrency type is <code>CONCUR_READ_ONLY</code> |
| <code>updatesAreDetected(int)</code>                               | Indicates whether or not a visible row update can be detected by calling the method <code>ResultSet.rowUpdated</code> .                               | False                                                                                                         |
| <b>JDBC 3.0</b>                                                    |                                                                                                                                                       |                                                                                                               |
| <code>supportsSavepoints()</code>                                  | Does the database support savepoints                                                                                                                  | False                                                                                                         |
| <code>supportsNamedParameters()</code>                             | Does the database support named parameters to callable statements                                                                                     | False                                                                                                         |
| <code>supportsMultipleOpenResults()</code>                         | Indicates whether it is possible to have multiple <code>ResultSet</code> objects returned from a <code>CallableStatement</code> object simultaneously | False                                                                                                         |
| <code>supportsGetGeneratedKeys()</code>                            | Indicates whether auto-generated keys can be retrieved after a statement has been executed                                                            | False                                                                                                         |
| <code>getSuperTypes(String, String, String)</code>                 | Gets a description of user-defined type hierarchies defined in a particular schema in this database                                                   | Empty <code>ResultSet</code> object                                                                           |
| <code>getSuperTables(String, String, String)</code>                | Gets a description of tables defined in a particular schema in this database                                                                          | Empty <code>ResultSet</code> object                                                                           |
| <code>getAttributes(String, String, String, String, String)</code> | Gets a description of the given attribute of the given type for a user-defined type that is available in the given schema and catalog                 | Empty <code>ResultSet</code> object                                                                           |

**Table 4-3: Return Values for *DatabaseMetaData* Methods**

| Method                                         | Description                                                                                                  | Returns                                                                                            |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>supportsResultSetHoldability(int)</code> | Does the database support the given result set holdability                                                   | True if result set holdability is <code>ResultSet.CLOSE_CURSORS_AT_COMMIT</code> , otherwise False |
| <code>getResultSetHoldability()</code>         | Gets the default holdability of this <code>ResultSet</code> object                                           | Always returns <code>ResultSet.CLOSE_CURSORS_AT_COMMIT</code>                                      |
| <code>getDatabaseMajorVersion()</code>         | Gets the major version number of the underlying database                                                     | returns 9                                                                                          |
| <code>getDatabaseMinorVersion()</code>         | Gets the minor version number of the underlying database                                                     | returns 0                                                                                          |
| <code>getJDBCMajorVersion()</code>             | Gets the major JDBC version number of this driver                                                            | returns 3                                                                                          |
| <code>getJDBCMinorVersion()</code>             | Gets the minor JDBC version number of this driver                                                            | returns 0                                                                                          |
| <code>getSQLStateType()</code>                 | Indicates whether the SQLStates returned by <code>SQLException.getSQLState</code> is X/Open SQL CLI or SQL99 | returns <code>sqlState99</code>                                                                    |
| <code>locatorsUpdateCopy()</code>              | Indicates whether updates made to LOB are made on a copy or directly to the LOB                              | "Driver does not support this"Exception                                                            |
| <code>supportsStatementPooling()</code>        | Does the database support statement pooling                                                                  | False                                                                                              |

### 4.3 ERROR MESSAGES

The error messages generated by the driver, along with associated `SQLSTATE` and `c-treeSQL` error code values, are documented in the *c-treeSQL SQL Reference Manual*, in Table B–2.



## Glossary

### A.1 TERMS

**add [an ODBC data source]**

Make a data source available to ODBC through the Add operation of the ODBC Administrator utility. Adding a data source tells ODBC where a specific database resides and which ODBC driver to use to access it. Adding a data source also invokes a setup dialog box for the particular driver so you can provide other details the driver needs to connect to the database.

**ADMIN**

The default owner name for all system tables in a c-treeSQL database. Users must qualify references to system tables as ADMIN.tablename.

**alias**

A temporary name for a table or column specified in the FROM clause of an SQL query expression. Also called correlation name. Derived tables and search conditions that join a table with itself must specify an alias. Once a query specifies an alias, references to the table or column must use the alias and not the underlying table or column name.

**applet**

A special kind of Java program whose compiled class files a Java-enabled browser can download from the Internet and run.

**ASCII**

(American Standard Code for Information Interchange) A 7-bit character set that provides 128 character combinations.

**bytecode**

Machine-independent code generated by the Java compiler and executed by the Java interpreter.

**candidate key**

Another term for unique key.

**cardinality**

Number of rows in a result table.

**Cartesian product**

Also called cross-product. In a query expression, the result table generated when a FROM clause lists more than one table but specifies no join conditions. In such a case, the result table

is formed by concatenating every row of every table with all other rows in all tables. Typically, Cartesian products are not useful and are slow to process.

**client**

Generally, in client/server systems, the part of the system that sends requests to servers and processes the results of those requests.

**collation**

The rules used to control how character strings in a character set compare with each other. Each character set specifies a collating sequence that defines relative values of each character for comparing, merging and sorting character strings.

**column alias**

An alias specified for a column. See alias.

**constraint**

Part of an SQL table definition that restricts the values that can be stored in a table. When you insert, delete, or update column values, the constraint checks the new values against the conditions specified by the constraint. If the value violates the constraint, it generates an error. Along with triggers, constraints enforce referential integrity by insuring that a value stored in the foreign key of a table must either be null or be equal to some value in the matching unique or primary key of another table.

**correlation name**

Another term for alias.

**cross product**

Another term for Cartesian product.

**data dictionary**

Another term for system catalog.

**data source**

See ODBC data source.

**derived table**

A virtual table specified as a query expression in the FROM clause of another query expression.

**driver manager**

See JDBC driver manager and ODBC driver manager.

**foreign key**

A column or columns in a table whose values must either be null or equal to some value in a corresponding column (called the primary key) in another table. Use the REFERENCES clause in the SQL CREATE TABLE statement to create foreign keys.

**form of use**

The storage format for characters in a character set. Some character sets, such as ASCII, require one byte (octet) for each character. Others, such as Unicode, use two bytes, and are called multi-octet character sets.



**input parameter**

In a stored procedure specification, an argument that an application must pass when it calls the stored procedure. In an SQL statement, a parameter marker in the statement string that acts as a placeholder for a value that will be substituted when the statement executes.

**interface**

In Java, a definition of a set of methods that one or more objects will implement. Interfaces declare only methods and constants, not variables. Interfaces provide multiple-inheritance capabilities.

**Java snippet**

See snippet.

**JDBC**

Java Database Connectivity: a part of the Java language that allows applications to embed standard SQL statements and access any database that implements a JDBC driver.

**JDBC driver**

Database-specific software that receives calls from the JDBC driver manager, translates them into a form that the database can process, and returns data to the application.

**JDBC driver manager**

A Java class that implements methods to route calls from a JDBC application to the appropriate JDBC driver for a particular JDBC URL.

**join**

A relational operation that combines data from two tables.

**main-memory storage system / storage manager**

A storage system and storage manager supplied with c-treeSQL. It provides a mechanism for implementations to store data in memory instead of on disk. By using the main-memory storage system for volatile data such as temporary sort tables and dynamic indexes, implementations can improve performance of many queries, such as joins.

**manager**

A main component of the SQL engine. The SQL engine includes several managers, including the SQL statement manager, parser, and optimizer.

**metadata**

Data that details the structure of tables and indexes in the c-tree Plus. The SQL engine stores metadata in the system catalog.

**octet**

A group of 8 bits. Synonymous with byte, and often used in descriptions of character-set encoding format.

**ODBC application**

Any program that calls ODBC functions and uses them to issue SQL statements. Many vendors have added ODBC capabilities to their existing Windows-based tools.

### **ODBC data source**

In ODBC terminology, a specific combination of a database system, the operating system it uses, and any network software required to access it. Before applications can access a database through ODBC, you use the ODBC Administrator to add a data source -- register information about the database and an ODBC driver that can connect to it -- for that database. More than one data source name can refer to the same database, and deleting a data source does not delete the associated database.

### **ODBC driver**

Vendor-supplied software that processes ODBC function calls for a specific data source. The driver connects to the data source, translates the standard SQL statements into syntax the data source can process, and returns data to the application. c-treeSQL includes an ODBC driver that provides access to the c-treeSQL Server.

### **ODBC driver manager**

A Microsoft-supplied program that routes calls from an application to the appropriate ODBC driver for a data source.

### **optimizer**

Within the SQL engine, the manager that analyzes costs and statistics associated with the statement and converts the relational algebra tree to the most efficient form for execution. The optimizer stores the trees for later use.

### **output parameter**

In a stored procedure specification, an argument in which the stored procedure returns a value after it executes.

### **package**

A group of related Java classes and interfaces, like a class library in C++. The Java development environment includes many packages of classes that procedures can import. The Java runtime system automatically imports the java.lang package. Stored procedures must explicitly import other classes by specifying them in the IMPORT clause of a CREATE PROCEDURE statement.

### **parameter marker**

A question mark (?) in a procedure call or SQL statement string that acts as a placeholder for an input or output parameter supplied at runtime when the procedure executes. The CALL statement (or corresponding ODBC or JDBC escape clause) use parameter markers to pass parameters to stored procedures, and the SQLStatement, SQLPreparedStatement, and SQLCursor objects use them within procedures.

### **primary key**

A subset of the fields in a table, characterized by the constraint that no two records in a table may have the same primary key value, and that no fields of the primary key may have a null value. Primary keys are specified in a CREATE TABLE statement.

### **procedure body**

In a stored procedure, the Java code between the BEGIN and END keywords of a CREATE PROCEDURE statement.

**procedure result set**

In a stored procedure, a set of data rows returned to the calling application. The number and data types of columns in the procedure result set are specified in the RESULT clause of the CREATE PROCEDURE statement. The procedure can transfer data from an SQL result set to the procedure result set or it can store data generated internally. A stored procedure can have only one procedure result set.

**procedure specification**

In a CREATE PROCEDURE statement, the clauses preceding the procedure body that specify the procedure name, any input and output parameters, any result set columns, and any Java packages to import.

**procedure variable**

A Java variable declared within the body of a stored procedure, as compared to a procedure input parameter or output parameter, which are declared outside the procedure body and are visible to the application that calls the stored procedure.

**query expression**

The fundamental element in SQL syntax. Query expressions specify a result table derived from some combination of rows from the tables or views identified in the FROM clause of the expression. Query expressions are the basis of SELECT, CREATE VIEW, and INSERT statements, and can be used in some expressions and search conditions.

**referential integrity**

The condition where the value stored in a database table's foreign key must either be null or be equal to some value in another table's the matching unique or primary key. SQL provides two mechanisms to enforce referential integrity: constraints specified as part of CREATE TABLE statements prevent updates that violate referential integrity, and triggers specified in CREATE TRIGGER statements execute a stored procedure to enforce referential integrity.

**repertoire**

The set of characters allowed in a character set.

**result set**

In a stored procedure, either an SQL result set or a procedure result set. More generally, another term for result table.

**result table**

A virtual table of values derived from columns and rows of one or more tables that meet conditions specified by an SQL query expression.

**row identifier**

Another term for tuple identifier.

**search condition**

The SQL syntax element that specifies a condition that is true or false about a given row or group of rows. Query expressions and UPDATE statements can specify a search condition. The search condition restricts the number of rows in the result table for the query expression or

UPDATE statement. Search conditions contain one or more predicates. Search conditions follow the WHERE or HAVING keywords in SQL statements.

### **selectivity**

The fraction of a table's rows returned by a query.

### **server**

Generally, in client/server systems, the part of the system that receives requests from clients and responds with results to those requests.

### **snippet**

In a stored procedure, the sequence of Java statements between the BEGIN and END keywords in the CREATE PROCEDURE (or CREATE TRIGGER) statement. The Java statements become a method in a class the SQL engine creates and submits to the Java compiler.

### **SQL diagnostics area**

A data structure that contains information about the execution status (success, error or warning conditions) of the most recent SQL statement. The SQL-92 standard specified the diagnostics area as a standardized alternative to widely varying implementations of the SQLCA. c-treeSQL supports both the SQLCA and the SQL diagnostics area. The SQL GET DIAGNOSTICS statement returns information about the diagnostics area to an application, including the value of the SQLSTATE status parameter.

### **SQL engine**

The core component of the c-treeSQL environment. The SQL engine receives requests from applications, processes them, and returns results. The SQL engine conveys requests to the c-tree Plus engine.

### **SQLCA**

SQL Communications area: A data structure that contains information about the execution status (success, error or warning conditions) of the most recent SQL statement. The SQLCA includes an SQLCODE field. The SQLCA provides the same information as the SQL diagnostics area, but is not compliant with the SQL-92 standard. c-treeSQL supports both the SQLCA and the SQL diagnostics area.

### **SQLCODE**

An integer status parameter whose value indicates the condition status returned by the most recent SQL statement. An SQLCODE value of zero means success, a positive value means warning, and a negative value means an error status. SQLCODE is superseded by SQLSTATE in the SQL-92 standard. Applications declare either SQLSTATE or SQLCODE, or both. SQL returns the status to SQLSTATE or SQLCODE after execution of each SQL statement.

### **SQL result set**

In a stored procedure, the set of data rows generated by an SQL statement (SELECT and, in some cases, CALL).

### **SQLSTATE**

A 5-character status parameter whose value indicates the condition status returned by the most recent SQL statement. SQLSTATE is specified by the SQL-92 standard as a replacement for

the SQLCODE status parameter (which was part of SQL-89). SQLSTATE defines many more specific error conditions than SQLCODE, which allows applications to implement more portable error handling. Applications declare either SQLSTATE or SQLCODE, or both. SQL returns the status to SQLSTATE or SQLCODE after execution of each SQL statement.

**stored procedure**

A snippet of Java source code embedded in an SQL CREATE PROCEDURE statement. The source code can use all standard Java features as well as use c-treeSQL-supplied Java classes for processing any number of SQL statements.

**system catalog**

Tables created by the SQL engine that store information about tables, columns, and indexes that make up the database. The SQL engine creates and manages the system catalog.

**system tables**

Another term for system catalog.

**tid**

Another term for tuple identifier.

**transaction**

A group of operations whose changes can be made permanent or undone only as a unit to protect against data corruption.

**trigger**

A special type of stored procedure that helps insure referential integrity for a database. Like stored procedures, triggers also contain Java source code (embedded in a CREATE TRIGGER statement) and use c-treeSQL Java classes. However, triggers are automatically invoked ("fired") by certain SQL operations (an insert, update, or delete operation) on the trigger's target table.

**trigger action time**

The BEFORE or AFTER keywords in a CREATE TRIGGER statement. The trigger action time specifies whether the actions implemented by the trigger execute before or after the triggering INSERT, UPDATE, or DELETE statement.

**trigger event**

The statement that causes a trigger to execute. Trigger events can be SQL INSERT, UPDATE, or DELETE statements that affect the table for which a trigger is defined.

**triggered action**

The Java code within the BEGIN END clause of a CREATE TRIGGER statement. The code implements actions to be completed when a triggering statement specifies the target table.

**tuple identifier**

A unique identifier for a tuple (row) in a table. The SQL scalar function ROWID and related functions return tuple identifiers to applications.

**Unicode**

A superset of the ASCII character set that uses two bytes for each character rather than ASCII's

7-bit representation. Able to handle 65,536 character combinations instead of ASCII's 128, Unicode includes alphabets for many of the world's languages. The first 128 codes of Unicode are identical to ASCII, with a second-byte value of zero.

### **unique key**

A column or columns in a table whose value (or combination of values) must be unique. Use the UNIQUE clause of the SQL CREATE TABLE statement to create unique keys. Unique keys are also called candidate keys.

### **URL**

In general, a Universal Resource Locator used to specify protocols and locations of items on the Internet. In JDBC, a database connection string in the form jdbc:subprotocol:subname. The c-treeSQL JDBC Driver format for database URLs is  
jdbc:ctree:6597@host\_name:db\_name.

### **view**

A virtual table that recreates the result table specified by a SELECT statement. No data is stored in a view, but other queries can refer to it as if it were a table containing data corresponding to the result table it specifies.

### **virtual table**

A table of values that is not physically stored in a database, but instead derived from columns and rows of other tables. SQL generates virtual tables in its processing of query expressions: the FROM, WHERE, GROUP BY and HAVING clauses each generate a virtual table based on their input.

### **virtual machine**

The Java specification for a hardware-independent and portable language environment. Java language compilers generate code that can execute on a virtual machine. Implementations of the Java virtual machine for specific hardware and software platforms allow the same compiled code to execute without modification.

# Tutorial Source Code

## B.1 INTRODUCTORY TUTORIAL

```
/* -----
JDBC_TUTORIAL1.java      "Public Domain JDBC Example".

This application will perform the following:
    1. Logon onto a session
    2. Add 1 table with some fields
    3. Populate the table with a few records
    4. Display the contents of the table
----- */

import java.net.URL;
import java.sql.*;
import java.io.*;

public class JDBC_Tutorial1 {

    static Connection conn;
    static Statement stmt;

    /*****
    *   main() - The main() function implements the concept of      *
    *           "Init, define, manage and you're done..."        *
    *****/
    public static void main(String[] args) {

        initialize ();
        define ();
        manage ();
        done ();
    }

    /*****
    *   initialize() - Perform the minimum requirement of logging onto *
    *                 the ctree JQL Server                             *
    *****/
    private static void initialize ()
    {
        System.out.println("INIT");
        try {
            // Load the driver class
            Class.forName ("ctree.jdbc.ctreeDriver");
        }
    }
}
```

```
        conn = DriverManager.getConnection
            ("jdbc:ctree:6597@localhost:myDatabase", "ADMIN", "ADMIN");

        // Create a SQL statement
        stmt = conn.createStatement();

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 *   define() - Create the table.
 *****/
private static void define ()
{
    System.out.println("DEFINE");
    String tblCreate = "CREATE TABLE CUSTMAST (" +
        "cm_custnum VARCHAR(5), " +
        "cm_custzip VARCHAR(10), " +
        "cm_custstate VARCHAR(3), " +
        "cm_custrating VARCHAR(2), " +
        "cm_name VARCHAR(48), " +
        "cm_custaddress VARCHAR(48), " +
        "cm_custcity VARCHAR(48))";

    try {
        stmt.executeUpdate(tblCreate);
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 *   manage() - This funtion illustrates the manage concept.
 *               Add, Display and Delete some records
 *****/
private static void manage ()
{
    System.out.println("MANAGE");
    Delete_Records ();    // start fresh
    Add_Records ();
    Display_Records ();
    Delete_Records ();
}

/*****
 *   done() - This function handles closing a connection to
 *           the database
 *****/
private static void done ()
{
    System.out.println("DONE");
}
```



```

        try {
            stmt.close();
            conn.close();
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }

    /*****
    *   Add_Records() - Insert an array of records into a table
    *****/
    private static void Add_Records ()
    {
        System.out.println("\tAdd Records");
        String sqlInsert = "INSERT INTO custmast VALUES ";
        String RecordData[] = {
            "('1000','92867','CA','1','Bryan Williams','2999'
            Regency','Orange')",
            "('1001','61434','CT','1','Michael Jordan','13'
            Main','Harford')",
            "('1002','73677','GA','1','Joshua Brown','4356'
            Cambridge','Atlanta')",
            "('1003','10034','MO','1','Keyon Dooling','19771 Park'
            Avenue','Columbia')" };

        try {
            /* Loop inserting 1 record at a time */
            for (int count = 0; count < 4; count++)
            {
                stmt.executeUpdate(sqlInsert + RecordData[count]);
            }
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }

    /*****
    *   Display_Records() - This function displays the contents of
    *                       the CUSTMAST table
    *****/
    private static void Display_Records ()
    {
        System.out.println("\tDisplay Records");
        try {

            // Execute a query
            ResultSet rs = stmt.executeQuery ("select * from custmast");

            // Fetch the data from the query result set
            while (rs.next()) {
                System.out.println("\t  " + rs.getString(1) + "    " +
                rs.getString(5));
            }

        } catch (Exception e) {

```

```
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Delete_Records() - This function deletes the contents of custmast *
 *****/
private static void Delete_Records ()
{
    System.out.println("\tDelete Records");
    try {

        stmt.executeUpdate("DELETE FROM custmast");

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}
}
```

## B.2 RELATIONAL MODEL AND INDEXING TUTORIAL

```
/* -----
JDBC_TUTORIAL2.java      "Public Domain JDBC Example".

A simple query example
----- */

import java.net.URL;
import java.sql.*;
import java.io.*;

public class JDBC_Tutorial2 {
    static Connection conn;
    static Statement stmt;
    static Statement stmt1;
    static Statement stmt2;
    static Statement stmt3;

    /*****
     *   main() - The main() function implements the concept of
     *           "Init, define, manage and you're done..."
     *****/
    public static void main(String[] args) {

        initialize ();
        define ();
        manage ();
        done ();
    }

    /*****
     *   initialize() - Perform the minimum requirement of logging onto
     *                 the ctree JQL Server
     *****/
}
```

```

*****/
private static void initialize ()
{
    System.out.println("INIT");
    try {
        // Load the driver class
        Class.forName ("ctree.jdbc.ctreeDriver");

        conn = DriverManager.getConnection
            ("jdbc:ctree:6597@localhost:myDatabase",
            "ADMIN", "ADMIN");

        // Create a SQL statement
        stmt = conn.createStatement();
        stmt1 = conn.createStatement();
        stmt2 = conn.createStatement();
        stmt3 = conn.createStatement();

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
*   define() -   Create the tables.                               *
*                                                         *
*****/
private static void define ()
{
    System.out.println("DEFINE");

    System.out.println("\tCreate Tables");

    /* a list of orders by customer */
    Create_OrderList_Table();

    /* specific items regarding orders */
    Create_OrderItem_Table();

    /* information about product items */
    Create_ItemMaster_Table();

    /* information about customers */
    Create_CustomerMaster_Table();
}

/*****
*   manage() - This funtion illustrates the manage concept.      *
*               In this example tables are populated and a simple *
*               query is implemented using the JDBC C API fucntions. *
*****/
private static void manage ()
{
    float totalPrice;

```

```
System.out.println("MANAGE");

Delete_Records ();

System.out.println("\tAdd Records");
Add_OrderList_Records();
Add_OrderItem_Records();
Add_ItemMaster_Records();
Add_CustomerMaster_Records();

Display_Records ();

/* Perform a query:
List Customer name and total dollar amount per order

name                amount
@@@@@@@@@@@@@@@@    $xx.xx

FOR each order in the ORDERLIST
    Fetch ordernum
    Fetch custnum
    Fetch name from CUSTMAST based on custnum index
    FOR each ordernum in ORDERITEM
        Fetch quantity
        Fetch itemnum
        Fetch item price from ITEMMASTER
    NEXT
NEXT
*/
System.out.println("\n\tPerform Query");

try {

    // row value out of range
    ResultSet rsOrderList = stmt.executeQuery
        ("SELECT * " + "FROM orderlist");

    // FOR each order in the ORDERLIST
    while (rsOrderList.next()) {
        ResultSet rsCustMast = stmt1.executeQuery ("SELECT * " +
            "FROM custmast " + "WHERE cm_custnum = '" +
            rsOrderList.getString(4) + "'");
        rsCustMast.next();
        ResultSet rsOrderItem = stmt2.executeQuery ("SELECT * " +
            "FROM orderitems " + "WHERE oi_ordernum = '" +
            rsOrderList.getString(3) + "'");
        totalPrice = 0;
        while (rsOrderItem.next()) {
            ResultSet rsItemMast = stmt3.executeQuery ("SELECT * " +
                "FROM itemmast " + "WHERE im_itemnum = '" +
                rsOrderItem.getString(4) + "'");
            rsItemMast.next();
            totalPrice += (rsItemMast.getFloat(2) *
```

```

        rsOrderItem.getShort(3));
    rsItemMast.close();
}

    System.out.println("\t " + rsCustMast.getString(5) + "    " +
        totalPrice);
    rsOrderItem.close();
    rsCustMast.close();
}
rsOrderList.close();

} catch (Exception e) {
    System.out.println("Exception: " + e.getMessage());
}
}

/*****
 * done() - This function handles closing a connection to the
 * database
 *****/
private static void done ()
{
    System.out.println("DONE");

    try {
        stmt.close();
        stmt1.close();
        stmt2.close();
        stmt3.close();
        conn.close();
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Display_Records() - This function displays the contents of
 * the CUSTMAST table
 *****/
private static void Display_Records ()
{
    System.out.println("\tDisplay Records");
    try {

        // Execute a query
        ResultSet rs = stmt.executeQuery ("select * from custmast");

        // Fetch the data from the query result set
        while (rs.next()) {
            System.out.println("\t " + rs.getString(1) + "    " +
                rs.getString(5));
        }
        rs.close();
    }
}

```

```
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }

    /*****
    * Delete_Records() - This function deletes the contents of custmast *
    *****/
    private static void Delete_Records ()
    {
        System.out.println("\n\tDelete Records");
        try {
            stmt.executeUpdate("DELETE FROM orderlist");
            stmt.executeUpdate("DELETE FROM orderitems");
            stmt.executeUpdate("DELETE FROM itemmast");
            stmt.executeUpdate("DELETE FROM custmast");

        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }

    /*****
    * Create the Order List Table *
    *****/
    private static void Create_OrderList_Table ()
    {
        String tblCreate = "CREATE TABLE orderlist (" +
            "ol_orderdate DATE, " +
            "ol_promdate DATE, " +
            "ol_ordernum VARCHAR(7), " +
            "ol_custnum VARCHAR(4))";

        try {
            stmt.executeUpdate(tblCreate);
            stmt.executeUpdate("CREATE INDEX custorder ON orderlist
                (ol_ordernum, ol_custnum)");
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }

    /*****
    * Create the Order Items Table *
    *****/
    private static void Create_OrderItem_Table ()
    {
        String tblCreate = "CREATE TABLE orderitems (" +
            "oi_ordernum VARCHAR(7), " +
            "oi_seqnumber SMALLINT, " +
            "oi_quantity SMALLINT, " +
            "oi_itemnum VARCHAR(6))";
    }
```

```

        try {
            stmt.executeUpdate(tblCreate);
            stmt.executeUpdate("CREATE INDEX orderitem ON orderitems
                               (oi_ordernum, oi_seqnumber)");
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }

    /*****
    * Create the Item Master Table
    *****/
    private static void Create_ItemMaster_Table ()
    {
        String tblCreate = "CREATE TABLE itemmast (" +
            "im_weight INTEGER, " +
            "im_price MONEY, " +
            "im_itemnum VARCHAR(6), " +
            "im_desc VARCHAR(48))";

        try {
            stmt.executeUpdate(tblCreate);
            stmt.executeUpdate("CREATE INDEX itemnum ON itemmast
                               (im_itemnum)");
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }

    /*****
    * Create the Customer Master Table
    *****/
    private static void Create_CustomerMaster_Table ()
    {
        String tblCreate = "CREATE TABLE custmast (" +
            "cm_custnum VARCHAR(5), " +
            "cm_zip VARCHAR(10), " +
            "cm_state VARCHAR(3), " +
            "cm_rating VARCHAR(2), " +
            "cm_name VARCHAR(48), " +
            "cm_address VARCHAR(48), " +
            "cm_city VARCHAR(48))";

        try {
            stmt.executeUpdate(tblCreate);
            stmt.executeUpdate("CREATE INDEX custnum ON custmast
                               (cm_custnum)");
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }

    /*****
    * Add records to our Order List Table
    *****/

```

```

*****/
private static void Add_OrderList_Records()
{
    String sqlInsert = "INSERT INTO orderlist VALUES ";
    String RecordData[] = {
        "('9/1/2002', '9/5/2002', '1', '1001')",
        "('9/2/2002', '9/6/2002', '2', '1002')";
    };

    try {

        /* Loop inserting 1 record at a time */
        for (int count = 0; count < 2; count++)
        {
            stmt.executeUpdate(sqlInsert + RecordData[count]);
        }

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Add records to our Order Item Table
 *****/
private static void Add_OrderItem_Records()
{
    String sqlInsert = "INSERT INTO orderitems VALUES ";
    String RecordData[] = {
        "('1', 1, 2, '1')",
        "('1', 2, 1, '2')",
        "('1', 3, 1, '3')",
        "('2', 1, 3, '3')";
    };

    try {

        /* Loop inserting 1 record at a time */
        for (int count = 0; count < 4; count++)
        {
            stmt.executeUpdate(sqlInsert + RecordData[count]);
        }

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Add records to our Item Master Table
 *****/
private static void Add_ItemMaster_Records()
{
    String sqlInsert = "INSERT INTO itemmast VALUES ";
    String RecordData[] = {
        "(10, 19.95, '1', 'Hammer')",
        "(3, 9.99, '2', 'Wrench')",
        "(4, 16.59, '3', 'Saw')";
    };

```



```

        "(1, 3.98, '4', 'Pliers')");
    }
    try {
        /* Loop inserting 1 record at a time */
        for (int count = 0; count < 4; count++)
        {
            stmt.executeUpdate(sqlInsert + RecordData[count]);
        }
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Add records to our Customer Master Table
 *****/
private static void Add_CustomerMaster_Records ()
{
    String sqlInsert = "INSERT INTO custmast VALUES ";
    String RecordData[] = {
        "('1000','92867','CA','1','Bryan Williams','2999 Regency','Orange')",
        "('1001','61434','CT','1','Michael Jordan','13 Main','Harford')",
        "('1002','73677','GA','1','Joshua Brown','4356 Cambridge','Atlanta')",
        "('1003','10034','MO','1','Keyon Dooling','19771 Park Avenue','Columbia')";
    };

    try {
        /* Loop inserting 1 record at a time */
        for (int count = 0; count < 4; count++)
        {
            stmt.executeUpdate(sqlInsert + RecordData[count]);
        }
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}
}

```

## B.3 LOCKING TUTORIAL

```

/* -----
   JDBC_TUTORIAL3.java      "Public Domain JDBC Example".
   Basic Record Locking example
   ----- */
import java.net.URL;
import java.sql.*;
import java.io.*;

public class JDBC_Tutorial3 {

```

```
        static Connection conn;
        static Statement stmt;
        static Statement stmt1;
        static Statement stmt2;

/*****
 *   main() - The main() function implements the concept of
 *           "Init, define, manage and you're done..."
 *****/
public static void main(String[] args) {

    initialize ();
    define ();
    manage ();
    done ();
}

/*****
 *   initialize() - Perform the minimum requirement of logging onto *
 *                 the ctree JQL Server
 *****/
private static void initialize ()
{
    System.out.println("INIT");
    try {
        // Load the driver class
        Class.forName ("ctree.jdbc.ctreeDriver");

        conn = DriverManager.getConnection
            ("jdbc:ctree:6597@localhost:myDatabase",
            "ADMIN", "ADMIN");

        // disable each SQL statement as a single transaction
        conn.setAutoCommit(false);

        // Create a SQL statement
        stmt  = conn.createStatement();
        stmt1 = conn.createStatement();
        stmt2 = conn.createStatement();
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 *   define() - Create the tables.
 *****/
private static void define ()
{
    System.out.println("DEFINE");

    // Create customer master table
    String tblCreate = "CREATE TABLE custmast (" +
        "cm_custnum VARCHAR(5), " +
```

```

        "cm_zip VARCHAR(10), " +
        "cm_state VARCHAR(3), " +
        "cm_rating VARCHAR(2), " +
        "cm_name VARCHAR(48), " +
        "cm_address VARCHAR(48), " +
        "cm_city VARCHAR(48)");

    try {
        stmt.executeUpdate(tblCreate);
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * manage() - Populate master tables then add orders as transactions *
 *****/
private static void manage ()
{
    System.out.println("MANAGE");

    Delete_Records ();

    System.out.println("\tAdd Records");
    Add_CustomerMaster_Records();

    Display_Records();    /* show contents of table */

    Update_CustomerMaster_Record();
    Display_Records();    /* show contents of table */
}

/*****
 * done() - This function handles closing a connection to
 *         the database
 *****/
private static void done ()
{
    System.out.println("DONE");
    try {
        stmt.close();
        stmt1.close();
        stmt2.close();
        conn.close();
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Update_CustomerMaster_Records() - Arbitrarily choose 1 record
 *                                  to update in order to demonstrate
 *                                  the effects of locking.
 *****/

```

```
private static void Update_CustomerMaster_Record()
{
    InputStreamReader isr = new InputStreamReader(System.in);
    BufferedReader stdin = new BufferedReader (isr);

    System.out.println("\tUpdate");

    try {
        stmt.executeUpdate("UPDATE custmast SET cm_name = 'KEYON
        DOOLING' where cm_custnum = '1003'");
        System.out.println("\tPress <Enter>");
        String input = stdin.readLine();
        conn.commit();

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 *   Display_Records() - This function displays the contents of
 *                       the CUSTMAST table
 *****/
private static void Display_Records ()
{
    System.out.println("\n\tDisplay CUSTMAST table");
    try {

        // Execute a query
        ResultSet rs = stmt.executeQuery ("select *
        from custmast");

        // Fetch the data from the query result set
        while (rs.next()) {
            System.out.println("\t  " + rs.getString(1) + "  " +
            rs.getString(5));
        }
        rs.close();

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Delete_Records() - This function deletes the contents of custmast *
 *****/
private static void Delete_Records ()
{
    System.out.println("\n\tDelete Records");
    try {

        stmt.executeUpdate("DELETE FROM custmast");
        conn.commit();
    }
}
```

```

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Add records to our Customer Master Table
 *****/
private static void Add_CustomerMaster_Records ()
{
    String sqlInsert = "INSERT INTO custmast VALUES ";
    String RecordData[] = {
        "('1000', '92867', 'CA', '1', 'Bryan Williams', '2999
        Regency', 'Orange')",
        "('1001', '61434', 'CT', '1', 'Michael Jordan', '13 Main',
        'Harford')",
        "('1002', '73677', 'GA', '1', 'Joshua Brown', '4356
        Cambridge', 'Atlanta')",
        "('1003', '10034', 'MO', '1', 'Keyon Dooling', '19771 Park
        Avenue', 'Columbia')" };

    try {

        /* Loop inserting 1 record at a time */
        for (int count = 0; count < 4; count++)
        {
            stmt.executeUpdate(sqlInsert + RecordData[count]);
            conn.commit();
        }
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}
}

```

## B.4 TRANSACTION PROCESSING TUTORIAL

```

/* -----
JDBC_TUTORIAL4.java      "Public Domain JDBC Example".
Transaction processing example
----- */
import java.net.URL;
import java.sql.*;
import java.io.*;

public class JDBC_Tutorial4 {

    static Connection conn;
    static Statement stmt;
    static Statement stmt1;
    static Statement stmt2;

    /*****

```

```
*   main() - The main() function implements the concept of           *
*           "Init, define, manage and you're done..."             *
*****/
public static void main(String[] args) {

    initialize ();
    define ();
    manage ();
    done ();
}

/*****
*   initialize() - Perform the minimum requirement of logging onto *
*               the ctree JQL Server                               *
*****/
private static void initialize ()
{
    System.out.println("INIT");
    try {
        // Load the driver class
        Class.forName ("ctree.jdbc.ctreeDriver");

        conn = DriverManager.getConnection
            ("jdbc:ctree:6597@localhost:myDatabase",
             "ADMIN", "ADMIN");
        conn.setAutoCommit(false);
        // Create a SQL statement
        stmt = conn.createStatement();
        stmt1 = conn.createStatement();
        stmt2 = conn.createStatement();

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
*   define() - Create the tables.                                     *
*****/
private static void define ()
{
    System.out.println("DEFINE");

    /* a list of orders by customer */
    Create_OrderList_Table();

    /* specific items regarding orders */
    Create_OrderItem_Table();

    /* information about product items */
    Create_ItemMaster_Table();

    /* information about customers */
    Create_CustomerMaster_Table();
}
```

```

}

/*****
 * manage() - Populate master tables then add orders as transactions  *
 *****/
private static void manage ()
{
    System.out.println("MANAGE");

    Delete_Records ();

    Add_ItemMaster_Records();
    Add_CustomerMaster_Records();

    Add_Transactions();

    Display_OrderList ();
    Display_OrderItems ();
}

/*****
 * done() - This function handles closing a connection to the      *
 *      database                                                    *
 *****/
private static void done ()
{
    System.out.println("DONE");
    try {
        stmt.close();
        stmt1.close();
        stmt2.close();
        conn.close();
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Add_Transactions() - Add item records associated with an order  *
 *      to the orderitems table. Verify the item exists in the    *
 *      itemmaster table. If it does not exist then rollback      *
 *      and abandon the transaction.                               *
 *                                                                 *
 *      Add the order record to the orderlist table and verify    *
 *      the customer number is a valid entry in the custmast     *
 *      table. If it exists then commit the transaction,          *
 *      otherwise rollback.                                       *
 *****/
private static void Add_Transactions()
{
    String[][] OrderData = {
        /* order date prom date ordernum custnum*/
        { "9/1/2002", "9/5/2002", "1", "1001"},
        { "9/2/2002", "9/6/2002", "2", "9999"},           // Bad custnum
    }
}

```

```
{ "9/2/2002", "9/6/2002", "3", "1002"} };
```

```
String[][] ItemData = {
/* ordernum  seqnum  qty  itemnum */
  { "1", "1", "2", "1" },
  { "1", "2", "1", "2" },
  { "2", "1", "1", "3" },
  { "2", "2", "3", "4" },
  { "3", "1", "2", "3" },
  { "3", "2", "2", "99"} // Bad itemnum
};

String      sql;
boolean     bTransFail;
int         iOrders, iItems = 0;

System.out.println("\tAdd Transaction Records... \n");
try {
  for(iOrders = 0; iOrders < 3; iOrders++)
  {
    bTransFail = false;
    while ( ItemData[iItems][0].compareTo
      (OrderData[iOrders][2]) == 0)
    {
      /* INSERT items associated with an order */
      sql = "INSERT INTO orderitems VALUES ( " +
        "'" + ItemData[iItems][0] + "'," +
        ItemData[iItems][1] + "," +
        ItemData[iItems][2] + "," +
        ItemData[iItems][3] + "' )";
      stmt.executeUpdate(sql);

      // Does this itemnum exist in the master
      sql = "SELECT im_itemnum FROM itemmast WHERE im_itemnum =
        '" + ItemData[iItems][3] + "'";
      ResultSet rsItemMast = stmt.executeQuery (sql);

      try {
        if (!(rsItemMast.next()))
        {
          conn.rollback();
          bTransFail = true;
          break;
        }
      } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
      }
      iItems++;
    }
  }

  if (!bTransFail)
  {
    // enter the order
    sql = "INSERT INTO orderlist VALUES ( " +
```



```

        "" + OrderData[iOrders][0] + "','" +
        OrderData[iOrders][1] + "','" +
        OrderData[iOrders][2] + "','" +
        OrderData[iOrders][3] + "''");
    stmt.executeUpdate(sql);
    sql = "SELECT cm_custnum FROM custmast WHERE cm_custnum =
    '" + OrderData[iOrders][3] + "''";
    ResultSet rsCustMast = stmt1.executeQuery (sql);
    if (rsCustMast.next())
        conn.commit();
    else
        conn.rollback();
    }
}
} catch (Exception e) {
    System.out.println("Exception: " + e.getMessage());
}
}

/*****
 *   Display_OrderList() - This function displays the contents of *
 *                       the ORDERLIST table                       *
 *****/
private static void Display_OrderList ()
{
    System.out.println("\n\tDisplay ORDERLIST table");
    try {

        // Execute a query
        ResultSet rs = stmt.executeQuery ("select * from
        orderlist");

        // Fetch the data from the query result set
        while (rs.next()) {
            System.out.println("\t " + rs.getDate(1) + " " +
            rs.getDate(2) + " " + rs.getString(3) + " " +
            rs.getString(4));
        }
        rs.close();

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 *   Display_OrderItems() - This function displays the contents of *
 *                       the ORDERITEMS table                       *
 *****/
private static void Display_OrderItems ()
{
    System.out.println("\n\tDisplay ORDERITEMS table");
    try {

```

```
// Execute a query
ResultSet rs = stmt.executeQuery ("select * from orderitems");

// Fetch the data from the query result set
while (rs.next()) {
    System.out.println("\t " + rs.getString(1) + " " +
        rs.getShort(2) + " " + rs.getShort(3) + " " +
        rs.getString(4));
}
rs.close();

} catch (Exception e) {
    System.out.println("Exception: " + e.getMessage());
}
}

/*****
 * Delete_Records() - This function deletes the contents of custmast *
 *****/
private static void Delete_Records ()
{
    try {

        stmt.executeUpdate("DELETE FROM orderlist");
        stmt.executeUpdate("DELETE FROM orderitems");
        stmt.executeUpdate("DELETE FROM itemmast");
        stmt.executeUpdate("DELETE FROM custmast");

    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Create the Order List Table *
 *****/
private static void Create_OrderList_Table ()
{
    String tblCreate = "CREATE TABLE orderlist (" +
        "ol_orderdate DATE, " +
        "ol_promdate DATE, " +
        "ol_ordnum VARCHAR(7), " +
        "ol_custnum VARCHAR(4))";

    try {
        stmt.executeUpdate(tblCreate);
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Create the Order Items Table *
 *****/
```

```

private static void Create_OrderItem_Table ()
{
    String tblCreate = "CREATE TABLE orderitems (" +
        "oi_ordernum VARCHAR(7), " +
        "oi_seqnumber SMALLINT, " +
        "oi_quantity SMALLINT, " +
        "oi_itemnum VARCHAR(6))";

    try {
        stmt.executeUpdate(tblCreate);
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Create the Item Master Table
 *****/
private static void Create_ItemMaster_Table ()
{
    String tblCreate = "CREATE TABLE itemmast (" +
        "im_weight INTEGER, " +
        "im_price MONEY, " +
        "im_itemnum VARCHAR(6), " +
        "im_desc VARCHAR(48))";

    try {
        stmt.executeUpdate(tblCreate);
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Create the Customer Master Table
 *****/
private static void Create_CustomerMaster_Table ()
{
    String tblCreate = "CREATE TABLE custmast (" +
        "cm_custnum VARCHAR(5), " +
        "cm_zip VARCHAR(10), " +
        "cm_state VARCHAR(3), " +
        "cm_rating VARCHAR(2), " +
        "cm_name VARCHAR(48), " +
        "cm_address VARCHAR(48), " +
        "cm_city VARCHAR(48))";

    try {
        stmt.executeUpdate(tblCreate);
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

```

```

/*****
 * Add records to our Item Master Table
 *****/
private static void Add_ItemMaster_Records()
{
    System.out.println("\tPopulate Item Master Table");
    String sqlInsert = "INSERT INTO itemmast VALUES ";
    String RecordData[] = {
        "(10, 19.95, '1', 'Hammer')",
        "(3, 9.99, '2', 'Wrench')",
        "(4, 16.59, '3', 'Saw')",
        "(1, 3.98, '4', 'Pliers')" };

    try {

        /* Loop inserting 1 record at a time */
        for (int count = 0; count < 4; count++)
        {
            stmt.executeUpdate(sqlInsert + RecordData[count]);
        }
        conn.commit();      // commit all activity to this point
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}

/*****
 * Add records to our Customer Master Table
 *****/
private static void Add_CustomerMaster_Records ()
{
    System.out.println("\tPopulate Customer Master Table");
    String sqlInsert = "INSERT INTO custmast VALUES ";
    String RecordData[] = {
        "('1000', '92867', 'CA', '1', 'Bryan Williams', '2999 Regency', 'Orange')",
        "('1001', '61434', 'CT', '1', 'Michael Jordan', '13 Main', 'Harford')",
        "('1002', '73677', 'GA', '1', 'Joshua Brown', '4356 Cambridge', 'Atlanta')",
        "('1003', '10034', 'MO', '1', 'Keyon Dooling', '19771 Park Avenue', 'Columbia')" };

    try {

        /* Loop inserting 1 record at a time */
        for (int count = 0; count < 4; count++)
        {
            stmt.executeUpdate(sqlInsert + RecordData[count]);
        }
        conn.commit();      // commit all activity to this point
    } catch (Exception e) {
        System.out.println("Exception: " + e.getMessage());
    }
}
}

```

## A

API, JDBC .....1-1  
Applet .....3-2  
Application server .....3-3  
    driver files .....3-3  
Architecture, JDBC .....1-1  
Authentication detail .....3-5

## B

Bridge drivers .....1-2

## C

Class files location .....3-2  
Class.forName .....3-4  
Connecting .....3-4  
Connecting to a database .....3-4  
Connection  
    example .....3-5  
Connection string .....3-5  
c-treeSQL JDBC driver .....1-3

## D

Database connection  
    example .....3-5  
Database connectivity .....3-4  
DatabaseMetaData .....4-2  
    return values .....4-2  
DhJDBCApplet.class .....3-2  
DhJDBCTest.jar .....3-2  
Driver files .....3-3  
Driver Manager  
    JDBC .....1-1  
Driver, types of .....1-2  
DriverManager.GetConnection .....3-4

## E

Environment variables .....3-3  
Error messages .....4-17

## G

getXXX .....4-1  
    method .....4-1

## I

Internet Explorer .....3-1

## J

JAR file .....3-2  
Java Archive file .....3-2  
Java URL connection string .....3-5  
java.sql .....1-1  
JavaSoft JDK Version 1.1.3 (Windows) ....3-1  
JavaSoft JDK Version 1.1.5 (UNIX) .....3-1  
JDBC .....1-1  
    applet .....3-1  
    application server .....3-3  
    architecture .....1-1  
    class files location .....3-2  
    comparison to ODBC .....1-3  
    connect to a database .....3-4  
    c-treeSQL JDBC driver .....1-3  
    driver files .....3-3  
    driver manager .....1-1  
    driver setup .....3-1  
    drivers .....1-2  
    environment variables .....3-3  
    error messages .....4-17  
    performance .....3-8  
    setup .....3-1  
    type 1 driver .....1-2  
    type 2 driver .....1-3

|                     |     |
|---------------------|-----|
| type 3 driver ..... | 1-3 |
| type 4 driver ..... | 1-3 |

## N

|                                                    |     |
|----------------------------------------------------|-----|
| Native-Method drivers .....                        | 1-3 |
| Native-Protocol All-Java drivers (c-treeSQL) ..... | 1-3 |
| Netscape .....                                     | 3-1 |
| Network-Protocol All-Java drivers .....            | 1-3 |

## O

|                        |     |
|------------------------|-----|
| Object                 |     |
| Properties .....       | 3-5 |
| ODBC                   |     |
| compared to JDBC ..... | 1-3 |

## P

|                                  |          |
|----------------------------------|----------|
| Performance improvements .....   | 3-8      |
| PreparedStatement.setXXX .....   | 4-1      |
| Program source code              |          |
| sample program source code ..... | 2-1, B-1 |
| Properties object .....          | 3-5      |

## R

|                         |     |
|-------------------------|-----|
| Required software ..... | 3-1 |
| Return values .....     | 4-2 |

## S

|                                            |          |
|--------------------------------------------|----------|
| Sample application .....                   | 2-1      |
| Setting up the c-treeSQL JDBC Driver ..... | 3-1      |
| setXXX .....                               | 4-1      |
| Source code example .....                  | 2-1, B-1 |

## T

|                              |     |
|------------------------------|-----|
| Transaction management ..... | 3-8 |
| Type 1 JDBC driver .....     | 1-2 |
| Type 2 JDBC driver .....     | 1-3 |

|                             |     |
|-----------------------------|-----|
| Type 3 JDBC driver .....    | 1-3 |
| Type 4 JDBC driver .....    | 1-3 |
| Types of JDBC drivers ..... | 1-2 |

## U

|                                  |     |
|----------------------------------|-----|
| UNIX                             |     |
| environment variables .....      | 3-3 |
| URL connection string .....      | 3-5 |
| User authentication detail ..... | 3-5 |

## V

|                       |     |
|-----------------------|-----|
| Virtual machine ..... | 3-1 |
|-----------------------|-----|

## W

|                             |     |
|-----------------------------|-----|
| Web page .....              | 3-2 |
| Web Server .....            | 3-1 |
| Windows NT                  |     |
| environment variables ..... | 3-3 |